L9: Market practice in strategy optimization, broker selection, and market tricks

Course Code: COMP7415

Lecturer: Tony Lam

Agenda

- Best practice to optimize a trading strategy
- Introduce common optimization methods
- Different type of investment funds
- The winning secret behind top tier hedge funds
- Guidelines to choose a trading broker
- Algo deployment on ALGOGENE
- Market tricks by traders and investment firms

Strategy Optimization

What is Strategy Optimization?

- Strategy optimization involves fine-tuning trading algorithms to maximize performance metrics like returns, Sharpe ratio, etc.
- Helps in improving trading strategies based on historical data, leading to better decision-making in live trading.

$$\underline{y} = f(\underline{x})$$

Strategy Optimization Process

1. Define the optimization objective
   - Commonly used performance metric
     - Max. annualized return
     - Max. Sharpe ratio
     - Min. maximal drawdown
   - User defined function (eg. Max. $F = \frac{(Annual\ return) \times (win\ rate)\sqrt{no.\ of\ trades}}{abs(VaR)}$)

Performance Statistics:

No. of Tradable Days:	31	No. of Win Days:	16	No. of Loss Days:	14
Win Rate:	53.3333%	Max. Consecutive Win Day:	5	Max. Consecutive Loss Day:	3
Odd Ratio:	1.1429	Max. Consecutive Gains:	457.88	Max. Consecutive Loss:	-461.27
No. of Trades:	2919	Average Consecutive Win Day:	0.93	Average Consecutive Loss Day:	0.6
Total PnL:	-173.62	Average Per Trade PnL:	-0.06	Average Per Day PnL:	-5.6
Mean Daily Return:	-0.0462%	Median Daily Return:	0.1058%	Mean Annual Return:	-11.6511%
Daily Return StdDev:	1.5529%	25th pencentile Daily Return:	-0.5793%	75th pencentile Daily Return:	0.5763%
Daily Return Downside StdDev:	1.0594%	95% 1 day return VaR:	-2.9652%	99% 1 day return VaR:	-4.128%
Daily Sharpe Ratio:	-0.0298	Annual Sharpe Ratio:	-0.4726	Max. Drawdown Amount:	995.9
Daily Sortino Ratio:	-0.0436	Annual Sortino Ratio:	-0.6928	Max. Drawdown Percent:	9.2023%
Max. Drawdown Duration:	8	Average Drawdown Duration:	2.45	Annual Volatility:	24.5542%
Gross Profit:	0.0	Gross Loss:	0.0	Profit Factor:	0.0
Jensen Alpha:	0.0	Beta:	0.0	Information Ratio:	0.0
Omega Ratio:	0.0	Treynor Ratio:	0.0	Tail Ratio:	0.8897
Calmar Ratio:	1.2661	Average Holding Day:	14.4259	Annual Turnover Rate:	0.0%

Strategy Optimization Process

2. Define the parameters
    - The control set of trading parameters (eg. take profit level, stop loss level, etc) in a trading strategy
    - Fewer parameters is preferred to reduce the chance of overfitting
3. Define optimization method
    - Common methods
        i. Brute force search / Grid search
        ii. Gradient descent
        iii. Genetic algorithm
    - Balance between accuracy and computing time
    - Some methods may not be appropriate due to violation of assumptions
4. Iterative backtesting and find the parameter set that achieve the optimal outcome

Brute force search

- Brute force search is the most straightforward method that finds the best solution by checking all possible solutions.
- Example: crack the password "12345"


```
import itertools, time

# Target password
target_password = "12345"

def brute_force_crack(target):
    # Start time for performance measurement
    start_time = time.time()

    # Generate all possible combinations of digits from 0 to 9
    for length in range(1, len(target) + 1):
        for attempt in itertools.product('0123456789', repeat=length):
            # Join the tuple to form the password attempt
            attempt_password = ''.join(attempt)
            print(f'Trying password: {attempt_password}’)

            # Check if the attempt matches the target password
            if attempt_password == target:
                end_time = time.time()
                print(f'Password cracked: {attempt_password}’)
                print(f'Time taken: {end_time - start_time:.2f} seconds’)
                return attempt_password

    print('Password not found.’)
    return None


brute_force_crack(target_password)
```

Gradient Descent

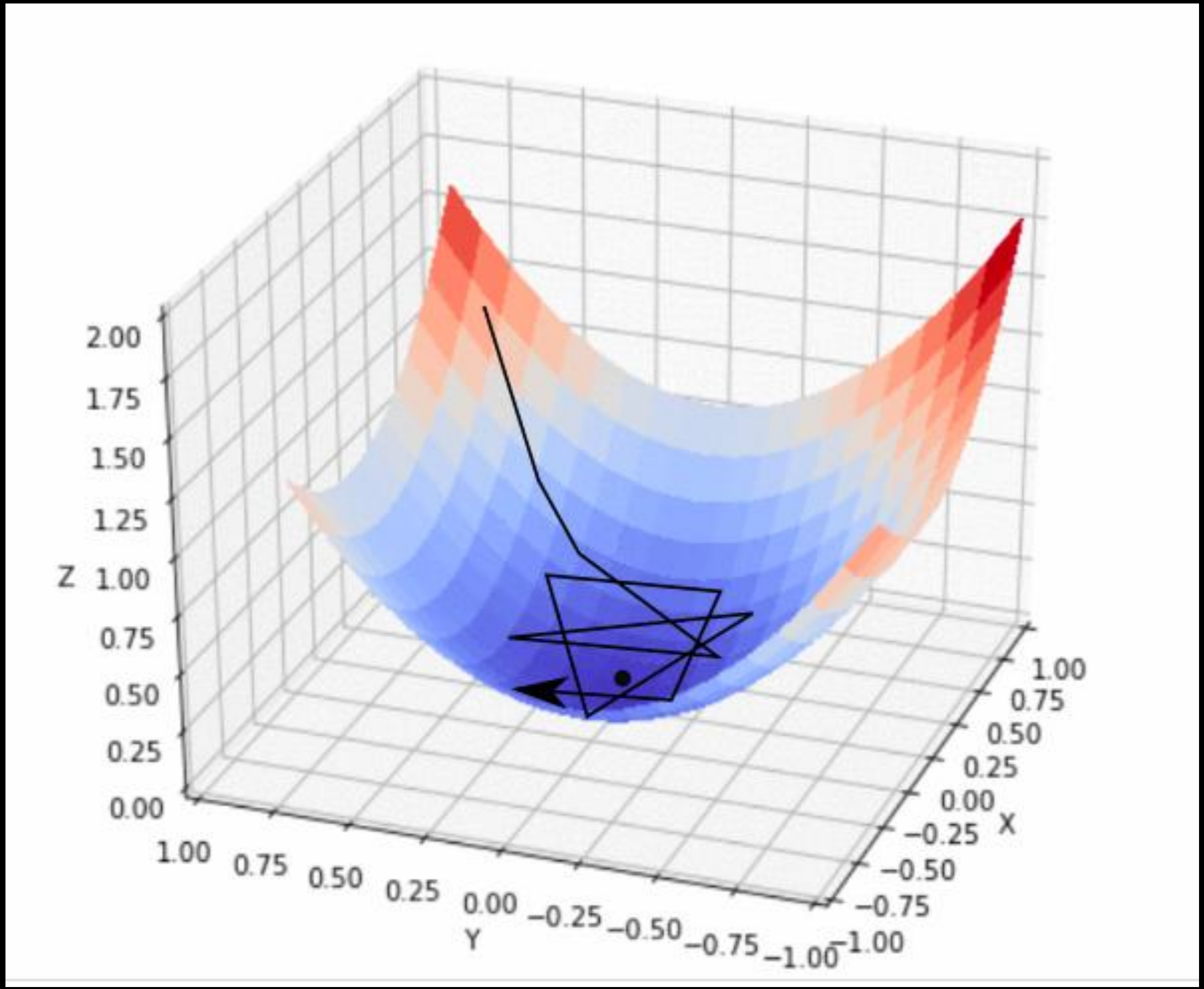
- Gradient descent is an optimization algorithm used to find the minimum of a function by iteratively moving toward the steepest descent.
- The update rule for gradient descent can be expressed as:

$$x_{n+1} = x_n - \alpha \cdot \nabla f(x_n)$$

where
- x_n is the current value.
- x_{n+1} is the next value.
- $\alpha > 0$ is the learning rate
- $\nabla f(x_n)$ is the gradient of the function at the current value. (i.e. 1st derivative of *f*)

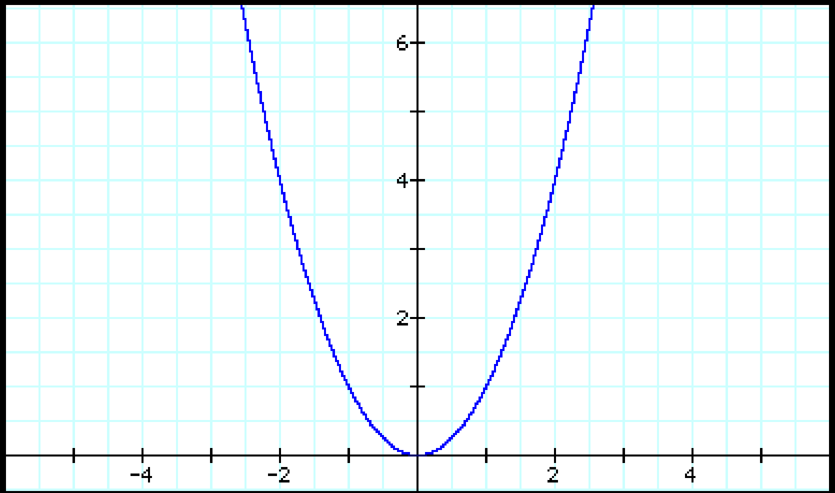
- Explanation
  - The gradient $\nabla f(x_n)$ indicates the direction of the steepest ascent. We move in the opposite direction to minimize the function.
  - The learning rate α controls how big of a step we take.
  - For finding the maximum, we can apply gradient descent method to $-f(.)$

Gradient Descent



Gradient Descent - Example

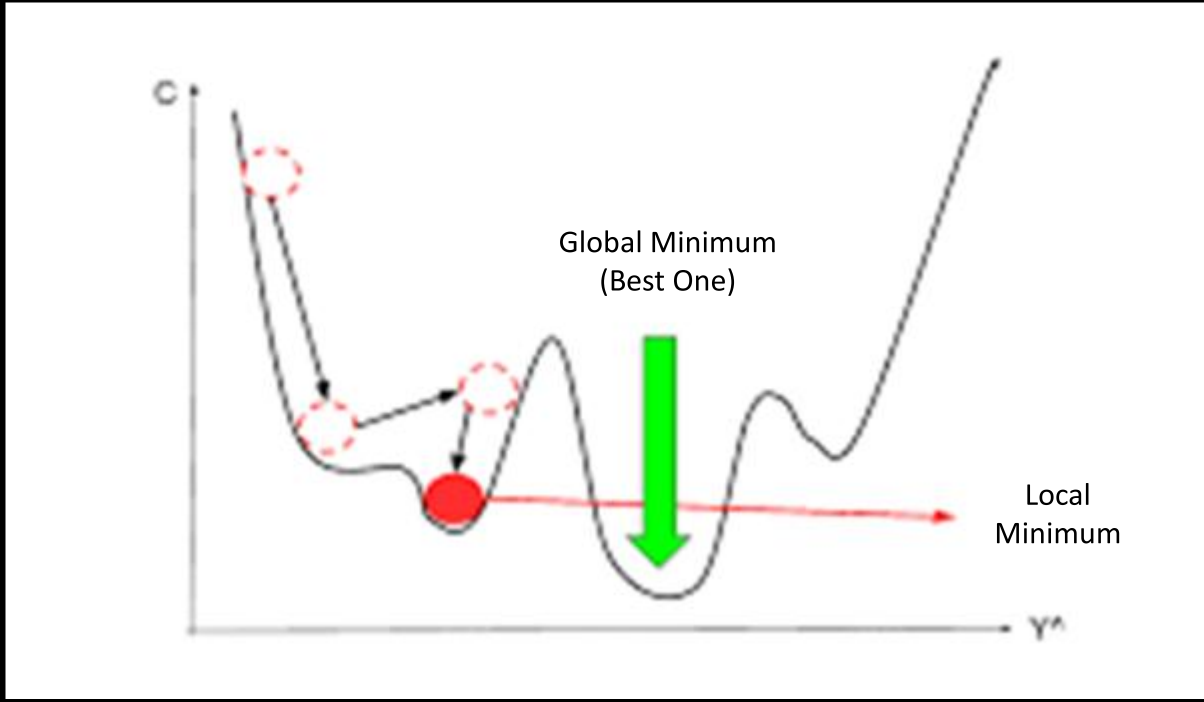
- Find the minimum of $f(x) = x^2$
- Thus, $\nabla f(x) = 2x$
- Suppose
  - $x_0 = 10$
  - $\alpha = 0.1$
  - Error tolerance level = 1e-6




```python
def gradient_descent_with_tolerance(f, df, x0, learning_rate, tolerance):
    x = x0 error = float('inf') # Initialize error
    while error > tolerance:
        x_new = x - learning_rate * df(x) # Update rule
        error = abs(x_new - x) # Calculate error
        x = x_new
    return x

# Function and its derivative
def f(x):
    return x**2

def df(x):
    return 2*x

# example
initial_x = 10
learning_rate = 0.1
tolerance = 1e-6
result = gradient_descent_with_tolerance(f, df, initial_x, learning_rate, tolerance)
print("Minimum value at x:", result) # Output: Close to 0
```

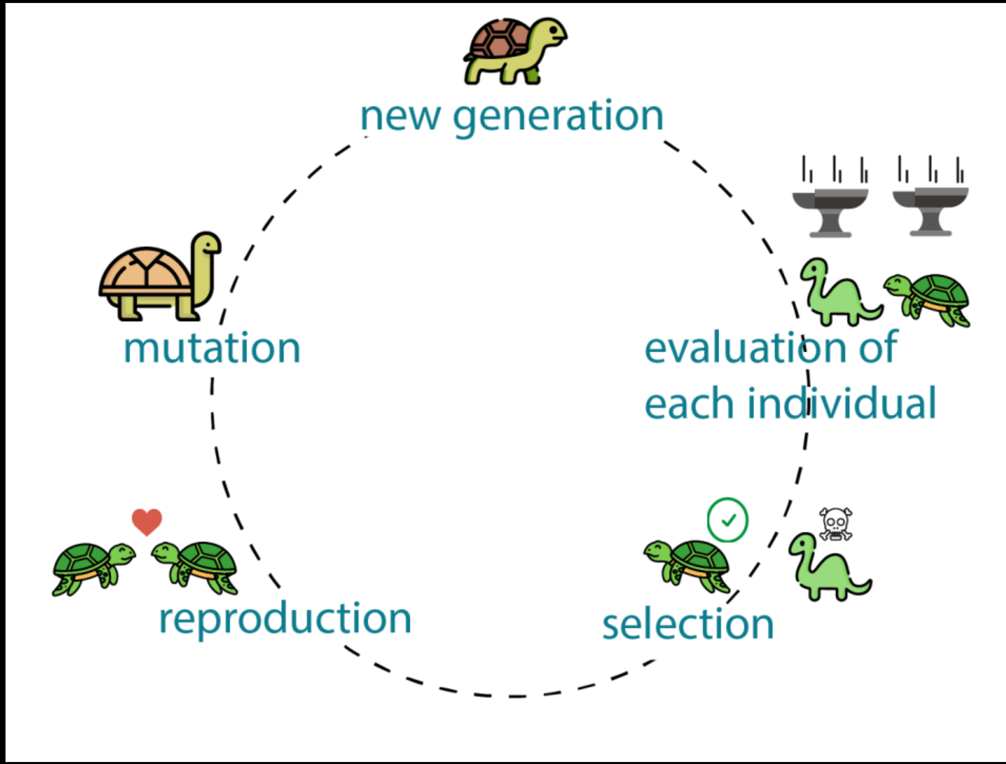
Gradient Descent - Limitation

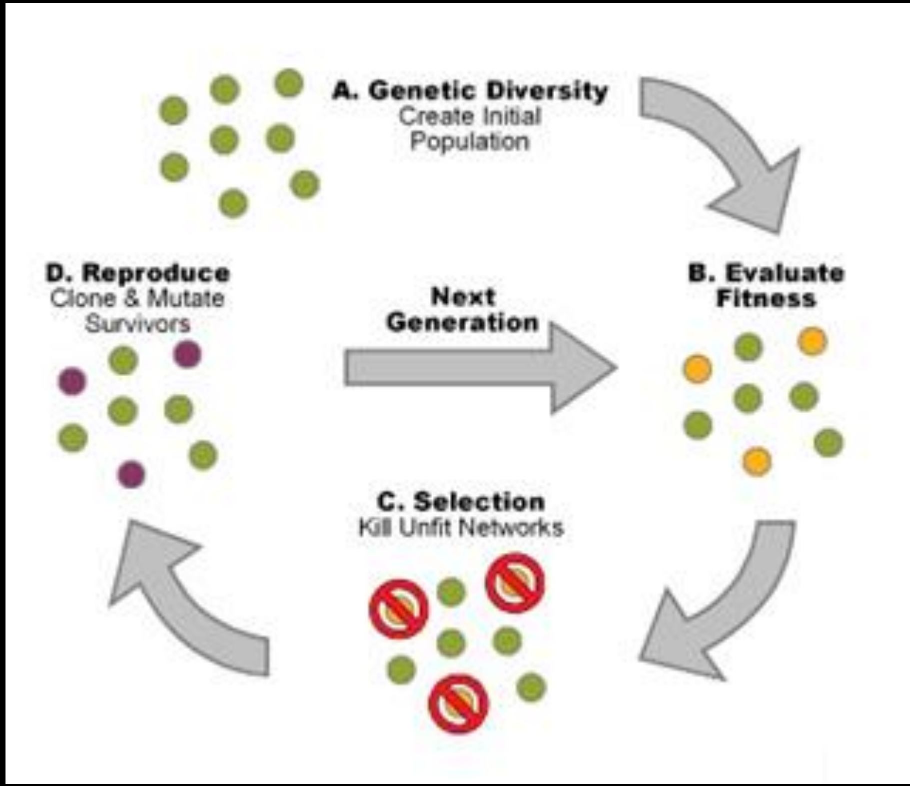
- The final solution is sensitive to the choice of
  - initial point x_0
  - learning rate α
- The optimization function is assumed to be continuous and differentiable
- No guarantee for convergence to the global optimal solution



Genetic Algorithm

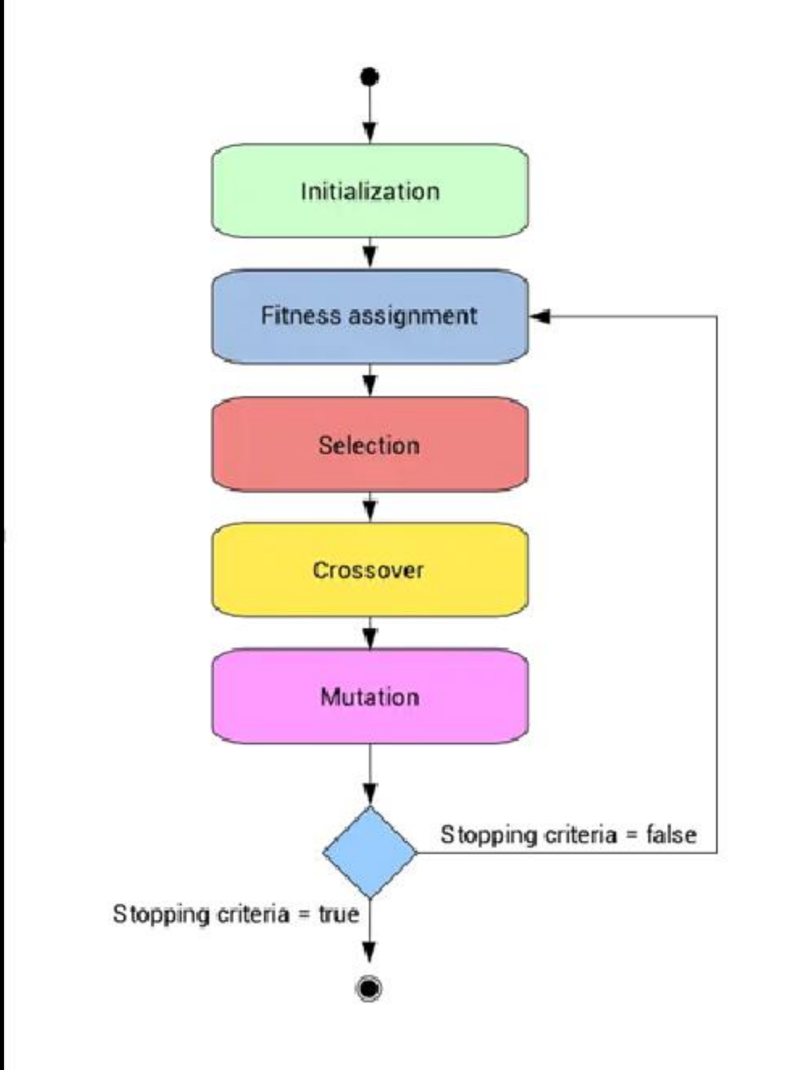
- Genetic algorithms are search heuristics inspired by natural selection and biological evolution that are used to solve optimization and search problems.



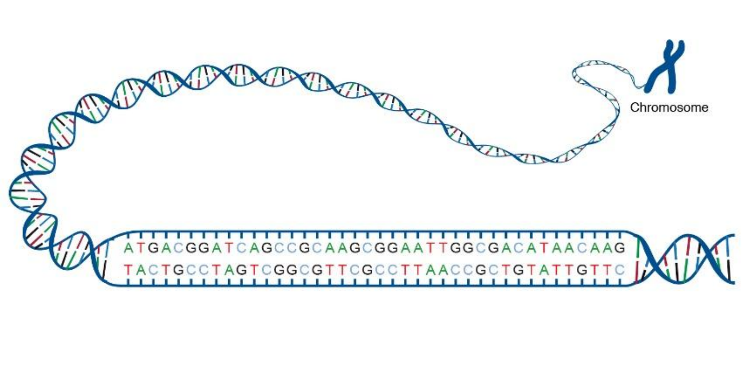


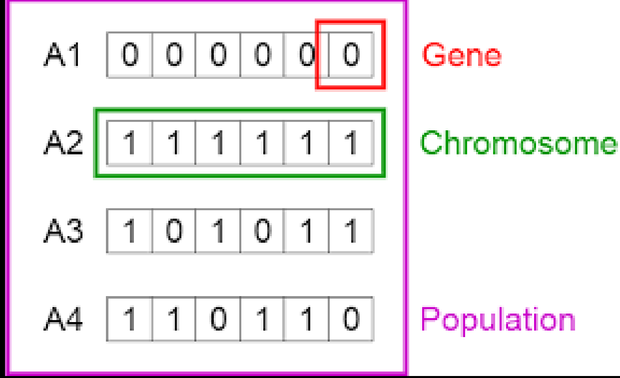
Steps Involved in a Genetic Algorithm

1. **Initialization**: Generate an initial population of potential solutions randomly.
2. **Fitness Evaluation**: Define a fitness function to evaluate how good each solution is.
3. **Selection**: Select individuals from the current population based on their fitness. Higher fitness individuals have a higher chance of being selected.
4. **Crossover**: Combine pairs of selected individuals (parents) to produce offspring (children). This introduces new solutions by mixing genetic information.
5. **Mutation**: Apply random changes to some individuals to maintain genetic diversity. This helps prevent premature convergence on suboptimal solutions.
6. **Replacement**: Replace the old population with the new one, often using strategies like elitism to retain the best solutions.
7. **Termination**: Repeat the process through multiple generations until a stopping criterion is met (eg. a satisfactory fitness level or a maximum number of generations).



Bitwise Operations





- **& (Bitwise AND)**: This operator compares each bit of two numbers. If both corresponding bits are 1, the resulting bit is set to 1; otherwise, it is set to 0.

```
a = 5 # (binary: 0101)
b = 3 # (binary: 0011)
result = a & b # (binary: 0001, decimal: 1)
```

- **<< (Left Shift)**: This operator shifts the bits of a number to the left by a specified number of positions. Each left shift effectively multiplies the number by 2.

```
a = 3 # (binary: 0011)
result = a << 1 # (binary: 0110, decimal: 6)
```

- **~ (Bitwise NOT)**:This operator inverts all the bits of a number. Each 0 becomes 1 and each 1 becomes 0.

```
a = 5 # (binary: 0101)
result = ~a # (binary: 1010, decimal: -6 in two's complement representation)
```

Genetic Algorithm - Example

- Maximizing the function $f(x) = x^2$ within the range [0, 31].

```
import random

def fitness(x):
    """Calculate the fitness of an individual."""
    return x**2 # Fitness function (maximize x^2)

def select(population):
    """Select two parents from the population based on fitness."""
    # Select two individuals using weights determined by their fitness
    return random.choices(population, weights=[fitness(x) for x in population], k=2)

def crossover(parent1, parent2):
    """Perform crossover between two parents to create two children."""
    point = random.randint(1, 4) # Random crossover point
    # Create children by mixing bits from parents
    child1 = (parent1 & (15 << point)) | (parent2 & ~(15 << point))
    child2 = (parent2 & (15 << point)) | (parent1 & ~(15 << point))
    return child1, child2

def mutate(x):
    """Mutate an individual with a small probability."""
    if random.random() < 0.1: # 10% mutation chance
        bit = random.randint(0, 4) # Select a random bit to mutate
        return x ^ (1 << bit) # Flip the selected bit
    return x # Return the original if no mutation occurs
```

```
# Example initialization
population = [random.randint(0, 31) for _ in range(10)] # Create a random population of integers
for _ in range(10): # Run for a fixed number of generations
    new_population = [] # Prepare a new population
    for _ in range(len(population) // 2): # Iterate for half the population size
        parent1, parent2 = select(population) # Select two parents
        child1, child2 = crossover(parent1, parent2) # Create two children

        print('debug ... ', parent1, parent2, child1, child2)

        # Mutate the children before adding them to the new population
        new_population.extend([mutate(child1), mutate(child2)]) # Update the population for the
next generation

    population = new_population # Find the best solution in the final population


best_solution = max(population, key=fitness)
print("Best solution:", best_solution) # Output: Close to 31
```

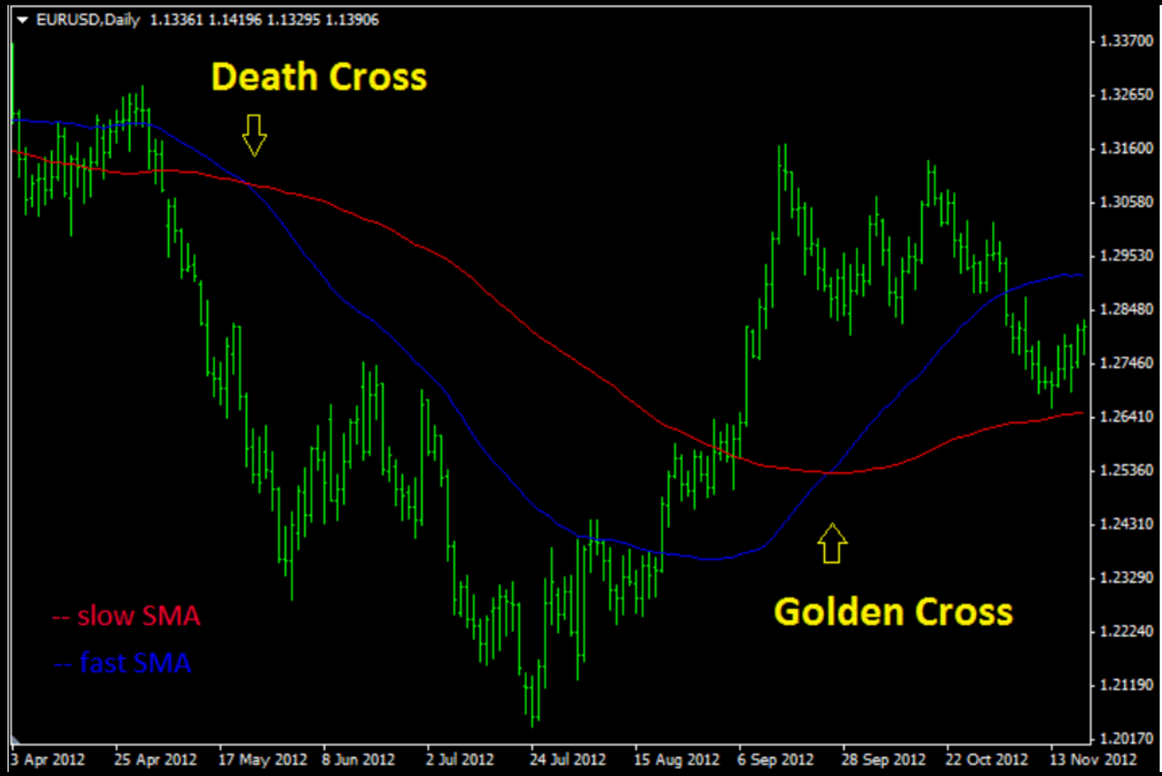
Genetic Algorithm: Remarks

- No Guarantee of Global Optimality
- Factors Influencing Convergence
  - Population size
  - Selection pressure
  - Crossover and mutation rates
  - The nature of the fitness landscape (eg. presence of local optima)
- Exploration vs Exploitation
  - Genetic algorithms balance exploration (searching through the solution space) and exploitation (refining existing solutions)
  - If exploration is too limited, the algorithm may converge prematurely to a local optimum

Optimize an MA Cross Strategy

MA Cross Strategy – Grid Search

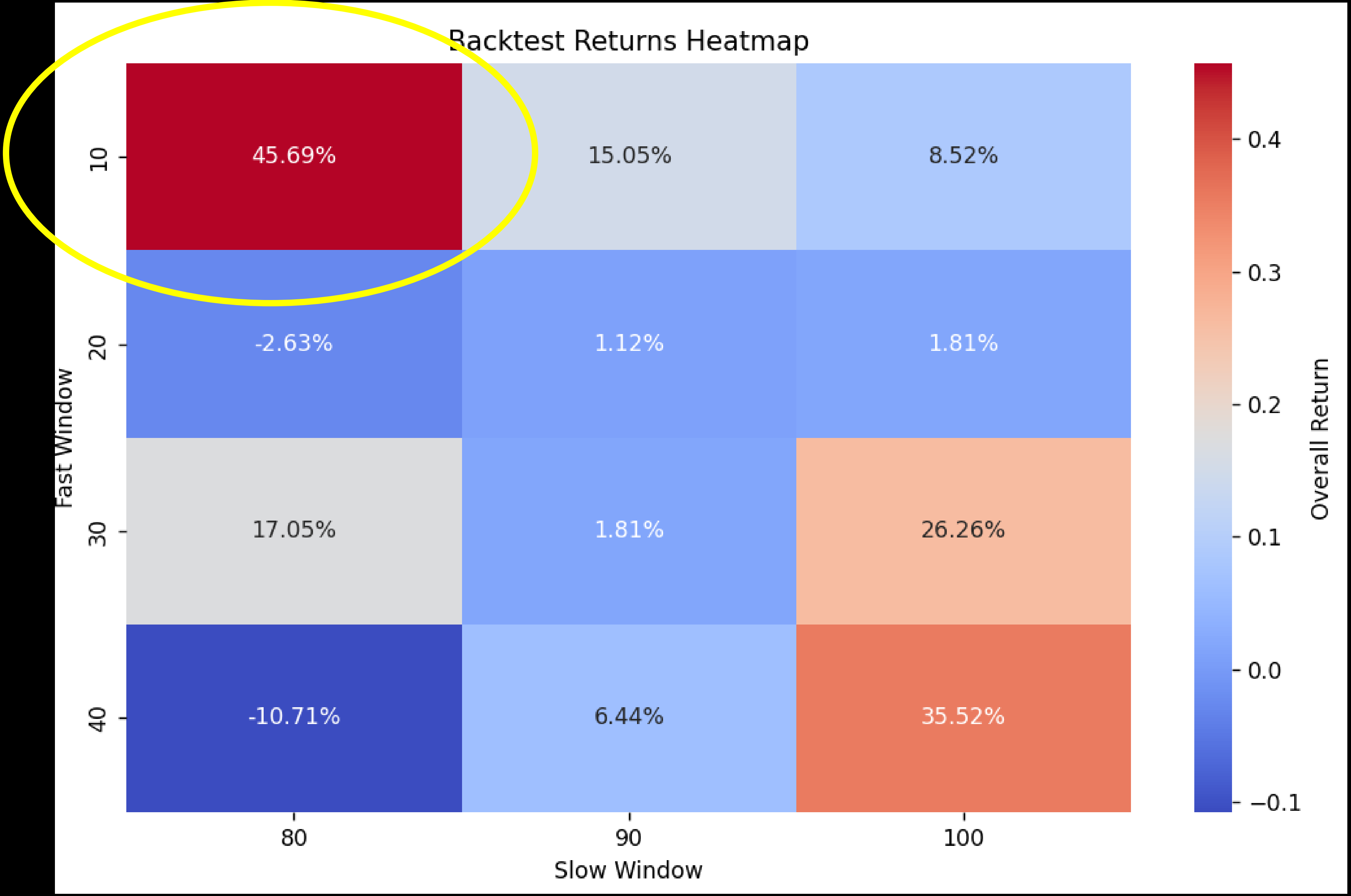
- Suppose we apply a MA Cross strategy on AAPL
- We are unsure what values of slow MA and fast MA to use (i.e. parameters = [fast_MA, slow_MA]), but we are interested in these ranges
  - fast_windows = [10, 20, 30, 40]
  - slow_windows = [80, 90, 100]
- We want to know which combination can achieve the highest overall return (i.e. objective = max. return)
- As there are only 12 combinations, we can simply run all the backtests for comparison. (i.e. method = brute force)



Optimize an MA Cross Strategy

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import yfinance as yf
from itertools import product

# Fetch historical data
data = yf.download('AAPL', start='2020-01-01', end='2023-01-01’)

# Define window parameters
fast_windows = [10, 20, 30, 40]
slow_windows = [80, 90, 100]

# Initialize a DataFrame to store results
results = pd.DataFrame(index=fast_windows, columns=slow_windows)
```

```
# Iterate through all combinations of fast_window and slow_window
for fast_window, slow_window in product(fast_windows, slow_windows):
    # Create a fresh copy of the data for each iteration
    temp_data = data.copy()

    # Create signals
    temp_data['fast_MA'] = temp_data['Close'].rolling(window=fast_window).mean()
    temp_data['slow_MA'] = temp_data['Close'].rolling(window=slow_window).mean()

    # Initialize the Signal column
    temp_data['Signal'] = 0

    # Create buy (1) and sell (-1) signals
    temp_data['Signal'] = np.where(temp_data['fast_MA'] > temp_data['slow_MA'], 1, 0)
    temp_data['Signal'] = np.where(temp_data['fast_MA'] < temp_data['slow_MA'], -1,
temp_data['Signal'])


    # Calculate returns
    temp_data['Market_Returns'] = temp_data['Close'].pct_change()
    temp_data['Strategy_Returns'] = temp_data['Market_Returns'] * temp_data['Signal'].shift(1)

    # Calculate cumulative returns
    total_return = (1 + temp_data['Strategy_Returns']).prod() - 1

    # Overall return from strategy # Store the result
    results.loc[fast_window, slow_window] = total_return


# Convert results to numeric type
results = results.astype(float)

# Plotting the heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(results, annot=True, fmt=".2%", cmap='coolwarm', cbar_kws={'label': 'Overall Return’})
plt.title('Backtest Returns Heatmap’)
plt.xlabel('Slow Window’)
plt.ylabel('Fast Window’)
plt.xticks(ticks=np.arange(len(slow_windows)) + 0.5, labels=slow_windows)
plt.yticks(ticks=np.arange(len(fast_windows)) + 0.5, labels=fast_windows)
plt.show()
```



Backtest Returns Heatmap
45.69%
15.05%
8.52%
-2.63%
1.12%
1.81%
17.05%
1.81%
26.26%
-10.71%
6.44%
35.52%
10
20
30
40
Fast Window
80
90
100
Slow Window
Overall Return
0.4
0.3
0.2
0.1
0.0
−0.1

MA Cross Strategy – Gradient Descent

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import yfinance as yf


data = yf.download('AAPL', start='2020-01-01', end='2023-01-01')

def compute_strategy_returns(fast_window, slow_window):
    temp_data = data.copy()
    temp_data['fast_MA'] = temp_data['Close'].rolling(window=fast_window).mean()
    temp_data['slow_MA'] = temp_data['Close'].rolling(window=slow_window).mean()
    temp_data['Signal'] = 0
    temp_data['Signal'] = np.where(temp_data['fast_MA'] > temp_data['slow_MA'], 1, temp_data['Signal'])
    temp_data['Signal'] = np.where(temp_data['fast_MA'] < temp_data['slow_MA'], -1, temp_data['Signal'])
    temp_data['Market_Returns'] = temp_data['Close'].pct_change()
    temp_data['Strategy_Returns'] = temp_data['Market_Returns'] * temp_data['Signal'].shift(1)
    return (1 + temp_data['Strategy_Returns']).prod() - 1


# Overall return # Gradient Descent parameters
learning_rate = 0.01
num_iterations = 1000
fast_window = 10
slow_window = 80
```

```python
# Gradient descent optimization
for _ in range(num_iterations):
    # Calculate returns for current windows
    current_return = compute_strategy_returns(fast_window, slow_window)

    # Small perturbations to estimate gradients
    fast_return_plus = compute_strategy_returns(fast_window + 1, slow_window)
    slow_return_plus = compute_strategy_returns(fast_window, slow_window + 1)

    # Estimate gradients
    fast_gradient = (fast_return_plus - current_return) / 1
    slow_gradient = (slow_return_plus - current_return) / 1

    # Update the windows
    fast_window += learning_rate * fast_gradient
    slow_window += learning_rate * slow_gradient

    # Ensure windows are within reasonable bounds
    fast_window = max(1, int(fast_window))
    slow_window = max(fast_window + 1, int(slow_window))

# Final optimized windows
print(f'Optimized Fast Window: {fast_window}')
print(f'Optimized Slow Window: {slow_window}')

# Calculate final returns
final_return = compute_strategy_returns(fast_window, slow_window)
print(f'Final Strategy Return: {final_return:.2%}')
```

```
Optimized Fast Window: 9
Optimized Slow Window: 79
Final Strategy Return: 30.09%
```

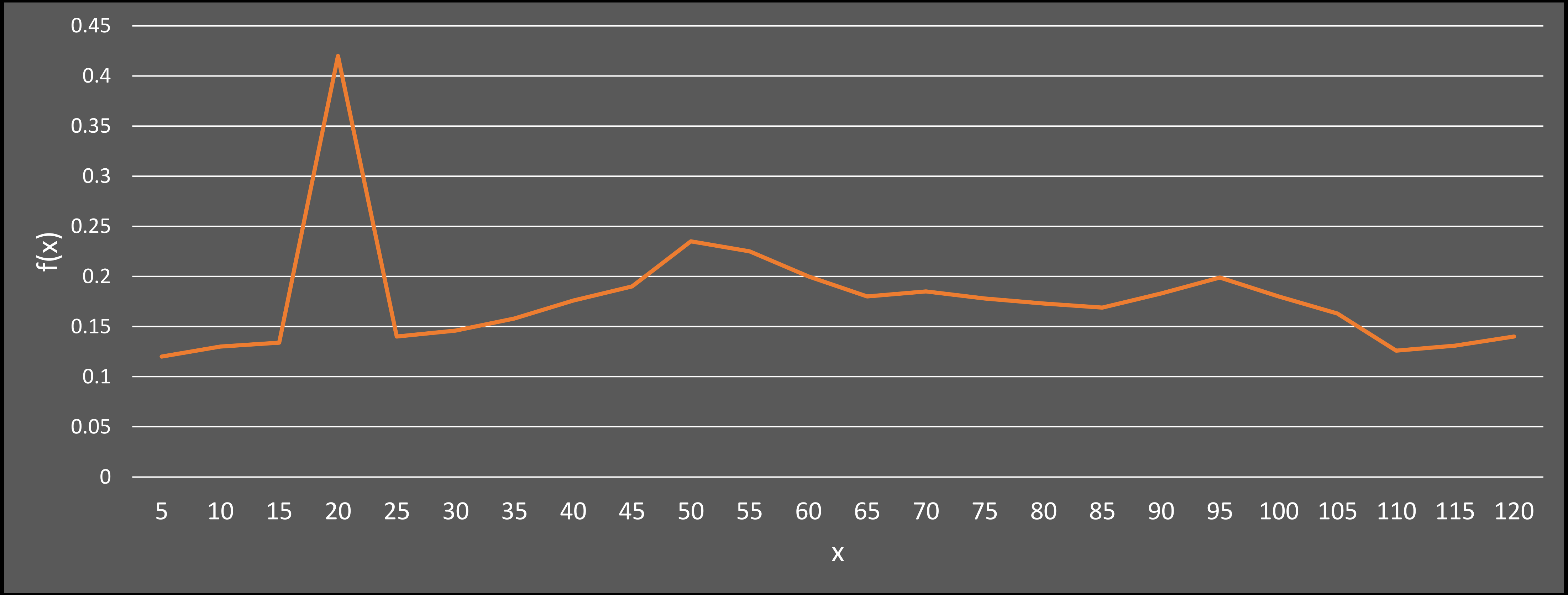
Is Gradient Decent a proper method?

- For the MA Cross strategy, the parameters can only be integers.
  - Slow MA period
  - Fast MA period
- Thus, it is a discrete function and therefore not differentiable.

Optimal vs Robust Solution

Optimal vs Robust Solution

- Suppose you obtain the following optimization results for a step size of 5.
- Which parameter X will you choose?



Optimal vs Robust Solution

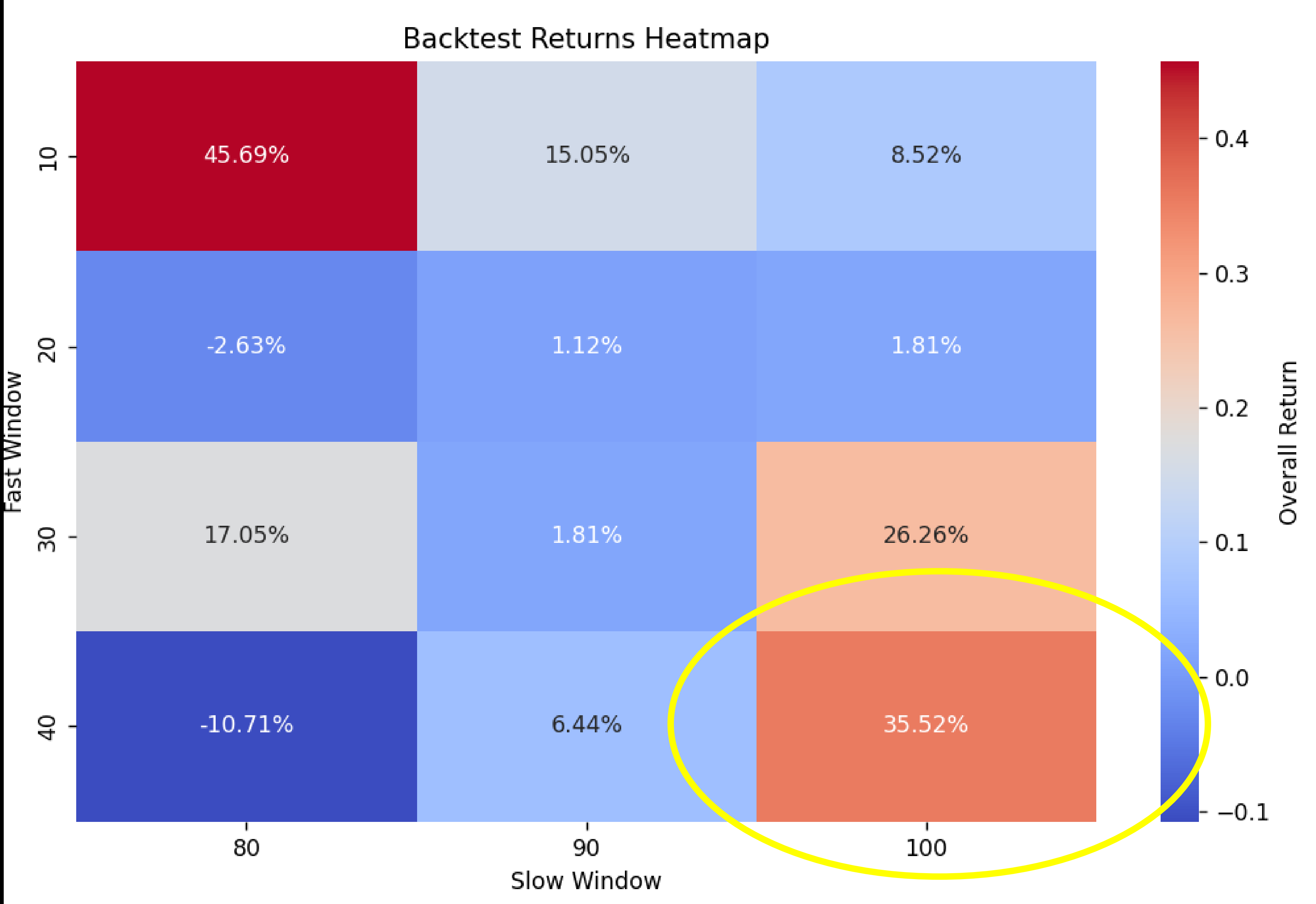
- For general data modelling,

	Optimality	Robustness
Pros	High performance in ideal scenarios	Consistent performance in diverse environments
Cons	Vulnerable to overfitting; may fail under new conditions	May sacrifice maximum performance for reliability

Optimal vs Robust Solution

- Optimal Strategy:
  - Good performance in backtesting but likely fails in live conditions.
- Robust Strategy:
  - Moderate performance but performs steadily across market conditions.

Choose a robust strategy!!!



Backtest Returns Heatmap
45.69%
15.05%
8.52%
-2.63%
1.12%
1.81%
17.05%
1.81%
26.26%
-10.71%
6.44%
35.52%
Fast Window
10
20
30
40
Slow Window
80
90
100
Overall Return
0.4
0.3
0.2
0.1
0.0
−0.1

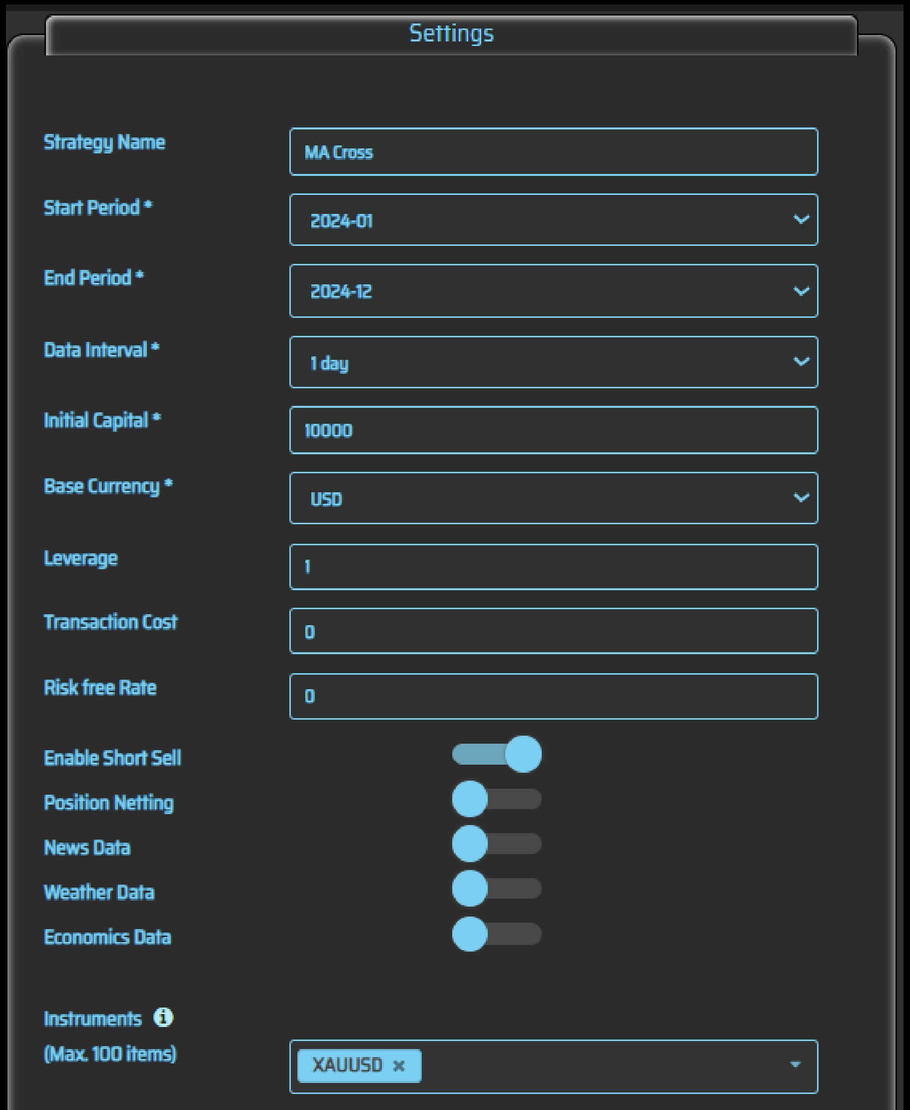
Guidelines to find a robust strategy

- Parameter Sensitivity Analysis
  - Test Parameter Ranges: Assess how performance varies with changes in strategy parameters.
  - Identify Stability Zones: Look for parameters that yield consistent results across a range of values.
- Walk-Forward Validation
  - Use a rolling window approach to optimize parameters on a subset of data while validating on the next segment.
  - For example,
    - Suppose you have 5 years of data
    - Split into 5 subsets, each with 1 year of data
    - Run the strategy with different parameter combinations over the 5 data subsets
    - Choose the parameter set that yield a moderate performance among the 5 data subsets

Strategy Optimization on ALGOGENE

Example

- We take the MA Cross strategy as an example
- Trading parameters are initially set to
  - Fast MA period = 7
  - Slow MA period = 14
- Backtest settings:
  - Period: 2024.01 – 2024.12
  - Instrument: XAUUSD
  - Data Interval: 1-day
  - Initial capital: 10,000 USD
  - Enable short-selling: True
  - Leverage: 1x
  - Transaction Cost: 0



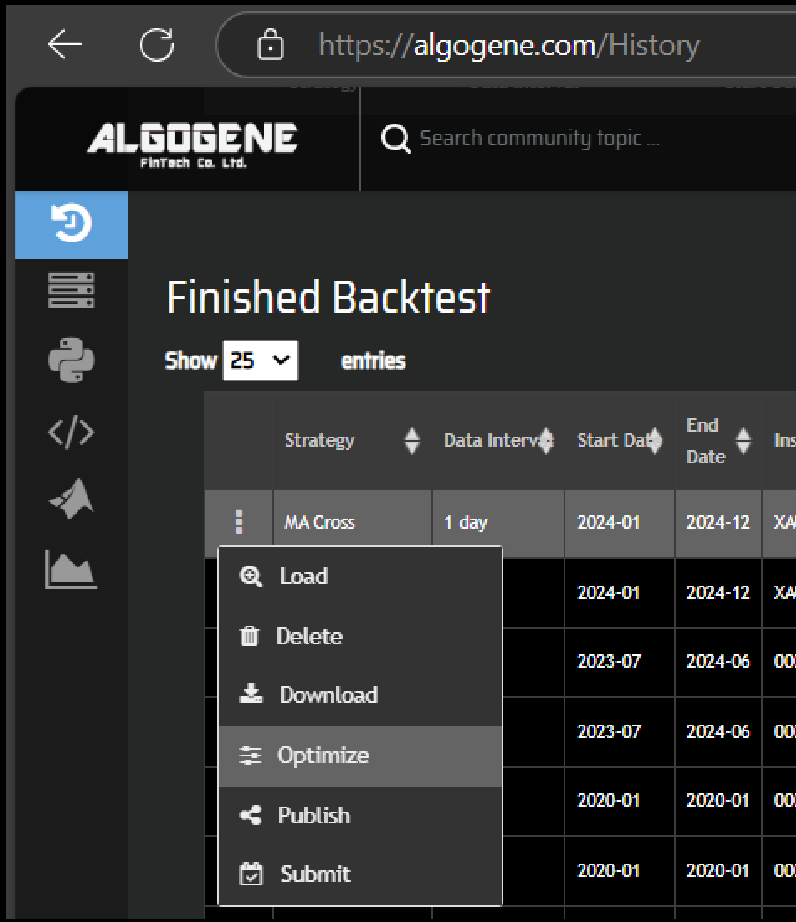
1. Define parameters at __init__

```
from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest
from datetime import datetime, timedelta
import talib, numpy

class AlgoEvent:
    def __init__(self):
        self.lasttradetime = datetime(1970,1,1)
        self.arr_close = numpy.array([])
        self.arr_fastMA = numpy.array([])
        self.arr_slowMA = numpy.array([])
        self.fastperiod = 7
        self.slowperiod = 14

    def start(self, mEvt):
        self.myinstrument = mEvt['subscribeList'][0]
        self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)
        self.evt.start()
```

```
    def on_bulkdatafeed(self, isSync, bd, ab):
        if bd[self.myinstrument]['timestamp'] >= self.lasttradetime + timedelta(hours=24):
            self.lasttradetime = bd[self.myinstrument]['timestamp']
            lastprice = bd[self.myinstrument]['lastPrice']
            self.arr_close = numpy.append(self.arr_close, lastprice)
            # keep the most recent observations
            if len(self.arr_close)>int(self.fastperiod+self.slowperiod):
                self.arr_close = self.arr_close[-int(self.fastperiod+self.slowperiod):]
            # fit SMA line
            self.arr_fastMA = talib.SMA(self.arr_close, timeperiod=int(self.fastperiod))
            self.arr_slowMA = talib.SMA(self.arr_close, timeperiod=int(self.slowperiod))
            # debug print result
            self.evt.consoleLog("arr_fastMA=", self.arr_fastMA)
            self.evt.consoleLog("arr_slowMA=", self.arr_slowMA)
            # check number of record is at least greater than both self.fastperiod, self.slowperiod
            if not numpy.isnan(self.arr_fastMA[-1]) and not numpy.isnan(self.arr_fastMA[-2]) and not
numpy.isnan(self.arr_slowMA[-1]) and not numpy.isnan(self.arr_slowMA[-2]):
                # send a buy order for Golden Cross
                if self.arr_fastMA[-1] > self.arr_slowMA[-1] and self.arr_fastMA[-2] < self.arr_slowMA[-
2]:
                    self.open_order(lastprice, 1, 'open')
                # send a sell order for Death Cross
                if self.arr_fastMA[-1] < self.arr_slowMA[-1] and self.arr_fastMA[-2] > self.arr_slowMA[-
2]:
                    self.open_order(lastprice, -1, 'open')

    def open_order(self, lastprice, buysell, openclose):
        order = AlgoAPIUtil.OrderObject(
            instrument = self.myinstrument,
            volume = 0.01,
            openclose = openclose,
            buysell = buysell,
            ordertype = 0 #0=market_order, 1=limit_order, 2=stop_order
        )
        self.evt.sendOrder(order)
```

Procedures

1. Successfully complete the backtest to make sure your script has no bug
2. Go to [My History], locate the strategy you want to optimize
3. Click “Optimize” from the drop-down



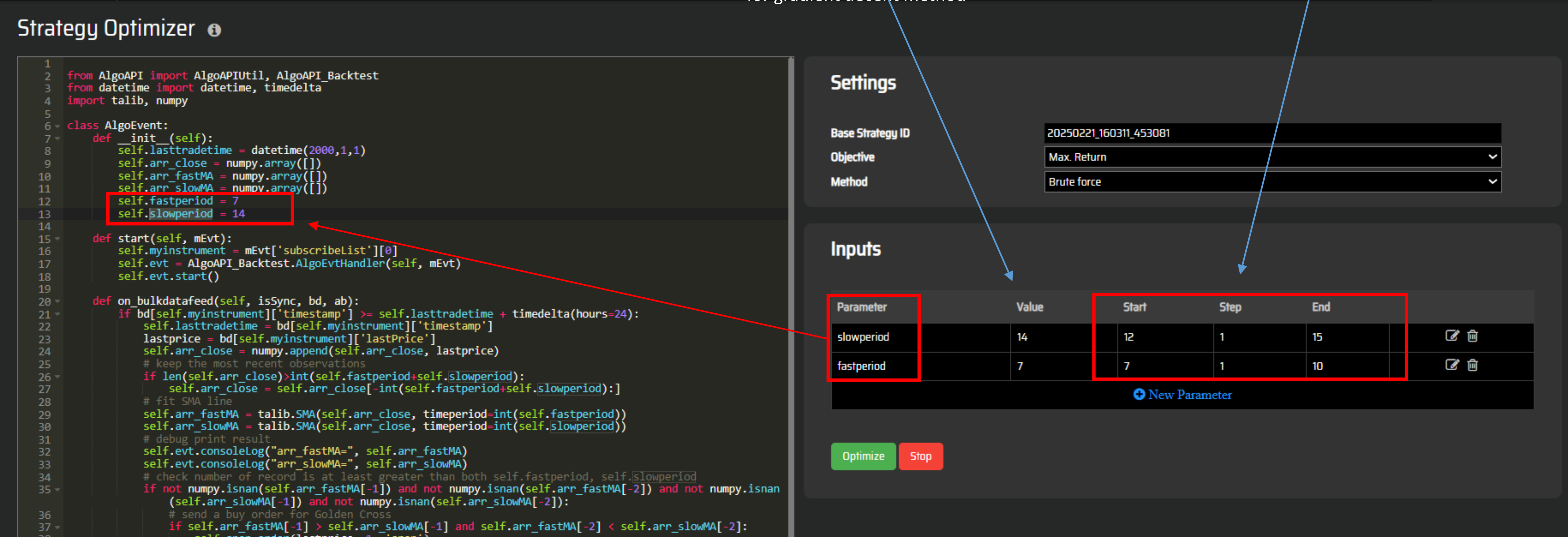
Reference: https://algogene.com/community/post/103

Procedures

4. Setup optimization objective, method, parameters and its ranges and step size

Initial estimate, only useful for gradient decent method

The range of parameters, including the start and end value



Procedures

5. Analyze results

Summary of all backtest iterations

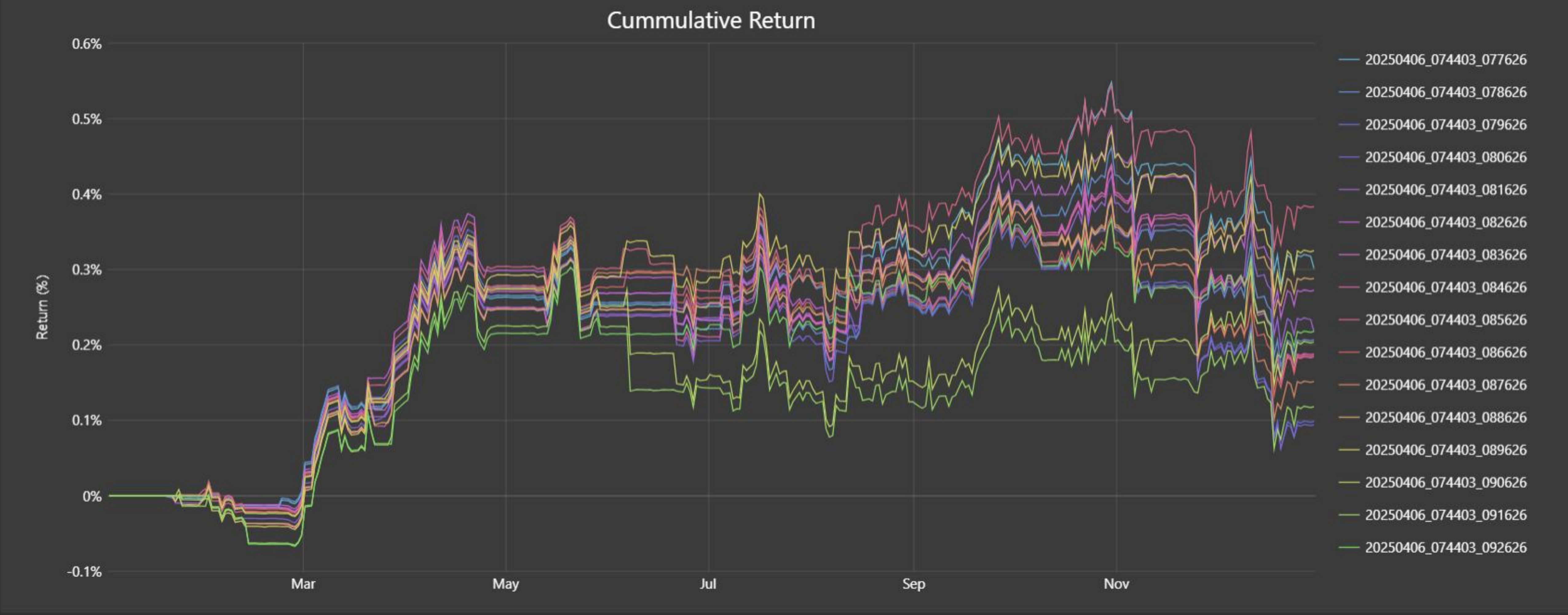
Results

100%

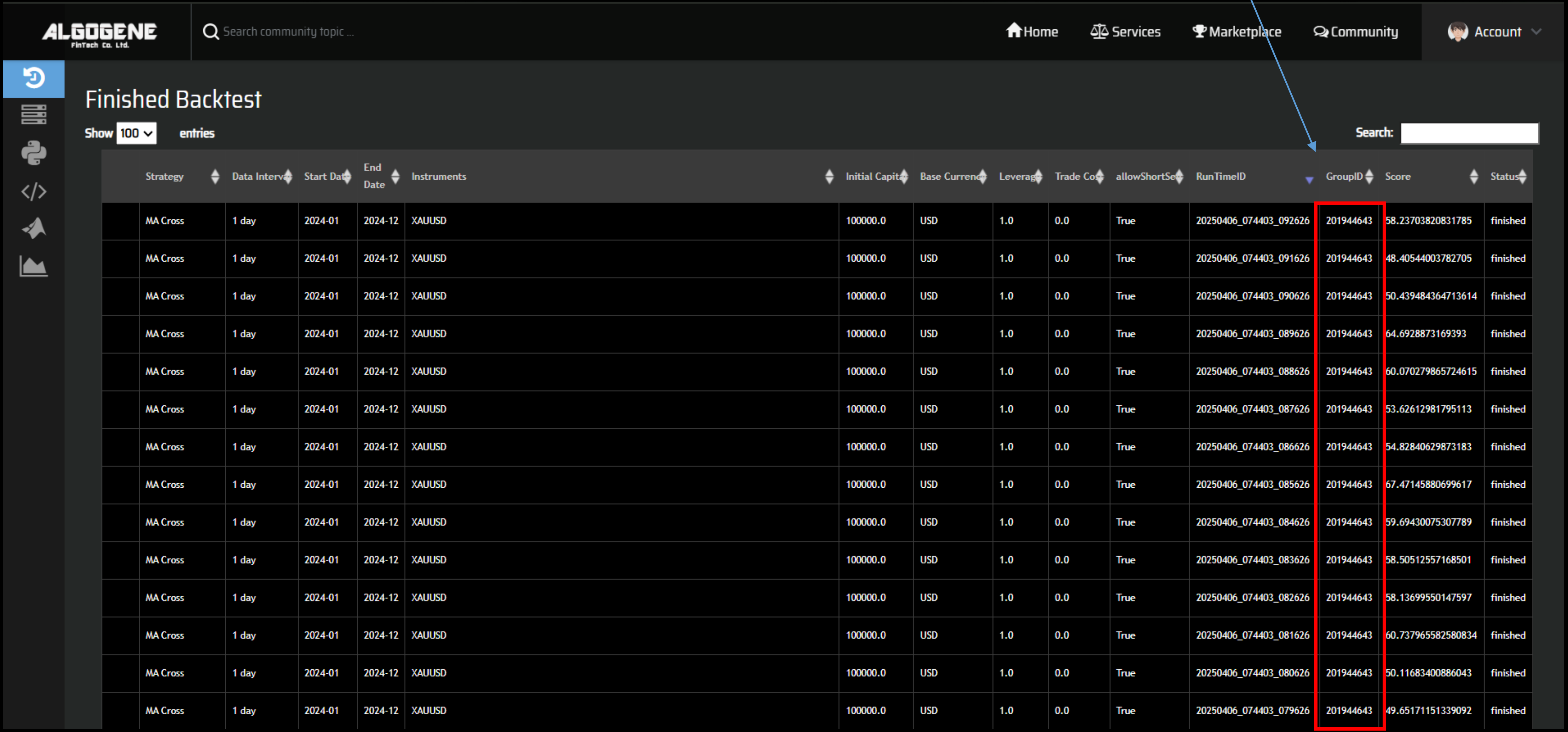
Summary Charts Performance

Pass	RuntimeID	Total Trades	Ann. Return (%)	Sharpe	Sortino	Max. Drawdown (%)	Volatility (%)	fastperiod	slowperiod
1	20250406_074403_077626	27	0.2558	0.8740	1.3013	0.2669	0.2915	7	12
2	20250406_074403_078626	24	0.1812	0.6183	0.9078	0.2904	0.2919	7	13
3	20250406_074403_079626	24	0.0869	0.2994	0.4332	0.3299	0.2892	7	14
4	20250406_074403_080626	20	0.0834	0.2862	0.4179	0.3262	0.2901	7	15
5	20250406_074403_081626	27	0.1848	0.6378	0.9307	0.2280	0.2885	8	12
6	20250406_074403_082626	24	0.2385	0.8038	1.1956	0.2525	0.2956	8	13
7	20250406_074403_083626	24	0.1655	0.5588	0.8253	0.2863	0.2950	8	14
8	20250406_074403_084626	22	0.1624	0.5511	0.8133	0.2844	0.2935	8	15
9	20250406_074403_085626	35	0.3263	1.0443	1.6014	0.2161	0.3112	9	12
10	20250406_074403_086626	28	0.1586	0.5326	0.7966	0.2422	0.2967	9	13
11	20250406_074403_087626	24	0.1334	0.4623	0.6817	0.2957	0.2874	9	14
12	20250406_074403_088626	24	0.2540	0.9278	1.4813	0.1699	0.2727	9	15
13	20250406_074403_089626	36	0.2768	0.8703	1.3379	0.2165	0.3168	10	12
14	20250406_074403_090626	32	0.1745	0.5750	0.8911	0.2471	0.3024	10	13
15	20250406_074403_091626	30	0.1018	0.3459	0.5297	0.2496	0.2930	10	14
16	20250406_074403_092626	24	0.1922	0.7062	1.1253	0.2146	0.2710	10	15

Summary Charts Performance



Each optimization task will generate a group of backtests with the same “GroupID”



Strategy	Data Interv	Start Dat	End Date	Instruments	Initial Capit	Base Curren	Leverag	Trade Co	allowShortSe	RunTimeID	GroupID	Score	Status
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_092626	201944643	58.23703820831785	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_091626	201944643	48.40544003782705	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_090626	201944643	50.439484364713614	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_089626	201944643	64.6928873169393	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_088626	201944643	60.070279865724615	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_087626	201944643	53.62612981795113	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_086626	201944643	54.82840629873183	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_085626	201944643	67.47145880699617	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_084626	201944643	59.69430075307789	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_083626	201944643	58.50512557168501	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_082626	201944643	58.13699550147597	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_081626	201944643	60.737965582580834	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_080626	201944643	50.11683400886043	finished
MA Cross	1 day	2024-01	2024-12	XAUUSD	100000.0	USD	1.0	0.0	True	20250406_074403_079626	201944643	49.65171151339092	finished

Investment Funds

Different Type of Investment Funds

There are several types of funds available in the market, each catering to different investment strategies and investor needs.

1. Exchange-Traded Funds (ETFs)
2. Mutual Funds
3. Money Market Funds
4. Private Equity Funds
5. Hedge Funds

1. Exchange-Traded Funds (ETFs)

- ETFs are traded on stock exchanges, similar to individual stocks, and typically track an index, commodity, or a basket of assets.
- Comprised of a diversified portfolio and can include stocks, bonds, or other securities.
- Key Features
  - **Liquidity**: Can be bought and sold throughout the trading day at market prices.
  - **Cost-Effective**: Generally have lower expense ratios compared to mutual funds.
  - **Flexibility**: Investors can use various trading strategies, including short selling and options.
- Examples:
  - SPDR S&P 500 ETF Trust (SPY): Tracks the S&P 500 index.
  - Invesco QQQ Trust (QQQ): Tracks the Nasdaq-100 Index, focusing on tech stocks.

Reference: https://en-rules.hkex.com.hk/rulebook/listing-fees-levies-and-trading-fees-new-issues-and-brokerage-0

2. Mutual Funds

- Investment vehicles that pool money from multiple investors to purchase a diversified portfolio of stocks, bonds, or other securities.
- Managed by professional fund managers who make investment decisions on behalf of the investors.
- Key Features
  - **Diversification**: Reduces risk by investing in a variety of assets.
  - **Liquidity**: Shares can be bought or sold on any business day.
  - **Accessibility**: Available to individual investors with relatively low minimum investments.
- Examples:
  - Fidelity Contrafund (FCNTX): Actively managed equity fund focusing on growth stocks.
  - T. Rowe Price Dividend Growth Fund (PRDGX): Invests in companies with a strong history of dividend growth.
  - Schwab Total Bond Market Fund (SWLBX): Provides exposure to a diversified portfolio of U.S. bonds.

Differences Between ETFs and Mutual Funds

	ETF	Mutual Funds
Trading	Traded on exchanges throughout the day	Bought/sold at end-of-day NAV
Management Style	Often passively managed	Can be actively or passively managed
Fees	Typically lower expense ratios	Higher fees, depending on management
Minimum Investment	No minimum (can buy 1 share)	Often requires a minimum investment

3. Money Market Funds

- A type of mutual fund that invests in short-term, high-quality debt instruments, such as treasury bills, commercial paper, and certificates of deposit.
- The objective is to provide liquidity, stability, and a modest return, making them a safe place to park cash.
- Key Features
    - **Liquidity**: Investors can typically access their funds quickly, often within a day.
    - **Safety**: Generally considered low-risk investments, as they invest in high-quality, short-term securities.
    - **Yield**: Offers higher interest rates than traditional savings accounts, though returns can vary based on market conditions.

4. Private Equity Funds

- Investment funds that provide capital to private companies or buyouts of public companies, aiming for long-term capital appreciation.
- Typically organized as limited partnerships, where the fund manager is the general partner and investors are limited partners.
- Key Features
  - **Investment Horizon**: Generally have a longer investment period (5-10 years) before realizing returns.
  - **Active Management**: Fund managers often take an active role in improving the operations and value of portfolio companies.
  - **High Returns**: Potential for significant returns, but also higher risk compared to traditional investments.

5. Hedge Fund

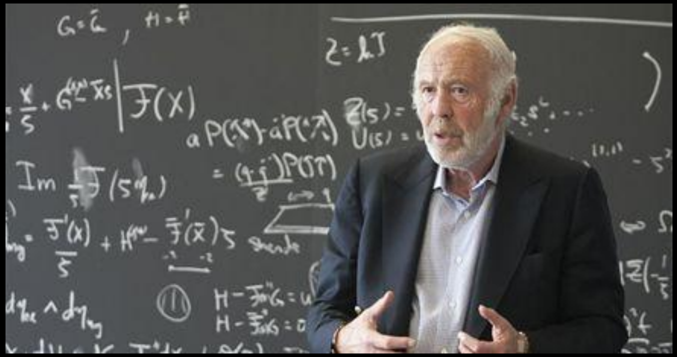
- Investment funds that employ diverse and advanced quant strategies to achieve high returns, often using leverage, derivatives, and short selling.
- Typically structured as limited partnerships, where fund managers are general partners and investors are limited partners.
- Could be open ended or closed ended fund
- Key Features
    - Flexibility: Can invest in a wide range of assets, including stocks, bonds, commodities, and real estate.
    - Risk Management: Utilize various strategies to hedge against market downturns and manage risk.
    - Accredited Investors: Generally only available to accredited, professional or institutional investors due to higher risks and complexity.

Summary of Investment Funds

Fund Type	Structure	Liquidity	Minimum Investment	Investment Strategy	Risk Level	Investor Type
Exchange-Traded Funds (ETFs)	Traded on exchanges, like stocks	Intraday	Varies (generally low)	Tracks an index, sector, or commodity	Moderate to high	Retail and institutional investors
Mutual Funds	Pooled investment vehicle	Daily (end of day)	Varies (often low)	Actively or passively managed, diversified	Moderate	Retail and institutional investors
Money Market Funds	Pooled investment vehicle	Daily	Varies (often low)	Invests in short-term, high-quality debt	Low	Conservative investors
Private Equity Funds	Limited partnerships	Illiquid (5-10 years)	High	Invests in private companies, buyouts	High	Accredited investors
Hedge Funds	Limited partnerships	Illiquid (varies)	High	Diverse strategies, including long/short, arbitrage	High	Accredited and institutional investors

Top Quant Trading Firms

- Renaissance Technologies
  - Founded by Jim Simons, Renaissance is renowned for its quantitative trading strategies, particularly through its Medallion Fund.
  - The Medallion Fund is known for achieving annualized returns of around **66% before fees since its inception in 1988**. However, the fund is closed to outside investors.
- Two Sigma Investments
  - Founded in 2001 by John Overdeck and David Siegel, Two Sigma employs advanced technology and data science to drive its investment strategies.
  - The firm has consistently delivered positive returns, with some funds reporting annualized returns of around 15-20% over the long term.



Jim Simons

John Overdeck, David Siegel

yahoo!finance

Virtu's IPO: First high-frequency trading firm to go public

Justine Underhill · Reporter
April 16, 2015

A the high-frequency trading debate continues, Virtu Financial (VIRT) sucessfully launched the first HFT firm IPO Wednesday with strong demand from investors.

The firm priced shares at $19 on Wednesday, the upper-end of its targeted range, and raised over $300 million at a $2.6 billion valuation.

This vote of confidence from investors comes just a year after Virtu pulled its IPO amid acute industry scrutiny, which followed the release of Michael Lewis' book "Flash Boys."

In a SEC filing last year, Virtu revealed it had only lost money on one day since 2009. "I thought it was a good thing to disclose to the world that the firm was profitable," said Doug Cifu, Virtu CEO, at a conference last June. "But, boy, did that backfire in my face, so I take responsibility for that." As of December 31, the 148-person firm generated net income of $190 million last year.

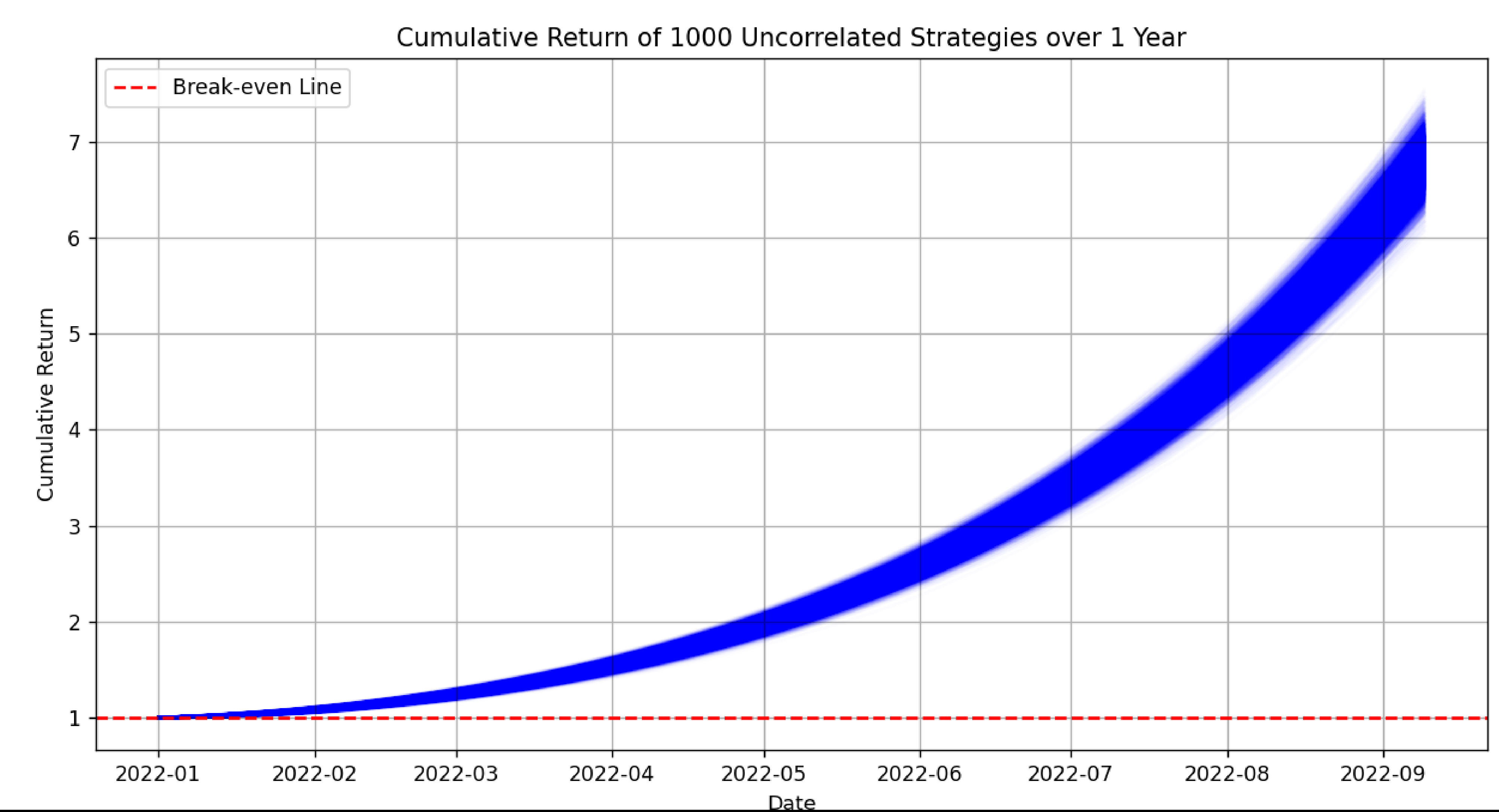
Reference: https://finance.yahoo.com/news/virtu-s-rewards--first-high-frequency-trading-firm-goes-public-151825635.html

The secret behind

- Hedge funds utilize multiple uncorrelated strategies to diversify risk and enhance overall performance.
- By combining strategies that react differently to market conditions, funds can stabilize returns and increase the likelihood of winning trades.
- Each strategy just has a winning rate slightly above 50%. Due to the law of large number, it can increase the probability of overall positive returns.

Example

```python
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Parameters
n_strategies = 1000 # Number of strategies
winning_rate = 0.55 # Each strategy has a winning rate of 55%
n_simulations = 10000 # Number of simulations
n_days = 252 # Trading days in a year # Simulate returns for each simulation
np.random.seed(0) # For reproducibility
all_cumulative_returns = np.zeros((n_simulations, n_days))

for sim in range(n_simulations):
    daily_returns = np.zeros((n_days, n_strategies))

    # Generate returns for each strategy
    for i in range(n_strategies):
        if np.random.rand() < winning_rate:
            daily_returns[:, i] = np.random.normal(0.01, 0.02, n_days) # Winning return
        else:
            daily_returns[:, i] = np.random.normal(-0.01, 0.02, n_days) # Losing return

    # Calculate cumulative returns for this simulation
    cumulative_returns = np.cumprod(1 + daily_returns, axis=0)
    all_cumulative_returns[sim, :] = np.mean(cumulative_returns, axis=1)

# Prepare date range
dates = pd.date_range(start='2022-01-01', periods=n_days)

# Plotting the results
plt.figure(figsize=(12, 6))
for sim in range(n_simulations):
    plt.plot(dates, all_cumulative_returns[sim, :], color='blue', alpha=0.01)
plt.title('Cumulative Return of 1000 Uncorrelated Strategies over 1 Year’)
plt.xlabel('Date’)
plt.ylabel('Cumulative Return’)
plt.axhline(y=1, color='red', linestyle='--', label='Break-even Line’)
plt.legend()
plt.grid()
plt.show()
```



Cumulative Return of 1000 Uncorrelated Strategies over 1 Year
Break-even Line
Cumulative Return
1
2
3
4
5
6
7
2022-01
2022-02
2022-03
2022-04
2022-05
2022-06
2022-07
2022-08
2022-09
Date

Broker Selection

Why Broker Choice Matters

- Profitability of your trading strategy
- Investor protection
- Key Considerations
  1. Trading costs
  2. Access to markets
  3. Trading platform features
  4. Regulatory compliance
  5. Customer support
  6. Reliability and reputation

Reference: https://fintechnews.hk/hong-kong-fintech-startup-ecosystem/

1. Evaluating Broker Fees

- Higher costs can erode profits, particularly for frequent traders
- Commission Structures:
  - Per trade vs flat fees
- Spread Costs:
  - Fixed vs variable spreads
- Other Fees:
  - Withdrawal fees, inactivity fees, data fees

UNITED STATES

Monthly Volume (shares)[1,7]	IBKR Pro - Tiered	IBKR Pro - Fixed	IBKR Lite
≤ 300,000	USD 0.0035	USD 0.005	USD 0.00[2]
300,001 - 3,000,000	USD 0.0020		
3,000,001 - 20,000,000	USD 0.0015		
20,000,001 - 100,000,000	USD 0.0010		
> 100,000,000	USD 0.0005		
Minimum per order (shares)	USD 0.35	USD 1.00	USD 0.00[3]
Minimum per order (fractional shares)	USD 0.01	USD 0.01	USD 0.00[3]
Maximum per order	1% of Trade Value[4,8]	1% of Trade Value[4,8]	USD 0.00
IB SmartRouting[SM]	✔	✔	✖
Eligibility	All	All	US Residents Only
Third Party Fees	• Regulatory Fees • Exchange Fees • Clearing Fees • Pass-Through Fees	• Regulatory Fees	• Regulatory Fees

Reference: https://www.interactivebrokers.com/en/pricing/commissions-home.php

2. Access to Markets

- Variety of Instruments and Asset Classes
  - Ability to trade in various markets increases opportunities.
  - Stocks, forex, commodities, crypto, etc
- Other things that may affects whether your strategy can be implemented
  - Account leverage
  - Account currency
  - Deposit method (eg. bank transfer, credit card, etc)

3. Trading Tools and Features

- **Platform Usability**
  - Intuitive interface can enhance trading efficiency
- **Analytical Tools**
  - Access to charts, indicators, and news feeds is useful for decision-making
  - API access for algorithmic trading
  - Real-time data feeds
  - Demo account to test the platform before risking real money
- **Mobile App**
  - Flexibility to trade on-the-go
  - Monitor the portfolio anytime, anywhere

4. Regulatory compliance

- Importance of Regulation
  - Ensures broker reliability and protects investors from fraud
  - Ensures transparency and fair trading practices
- Verify that the broker is regulated by a reputable authority such as
  - HKSFC (Hong Kong)
  - SEC, FINRA (U.S.)
  - FCA (U.K.)
  - ASIC (Australia)

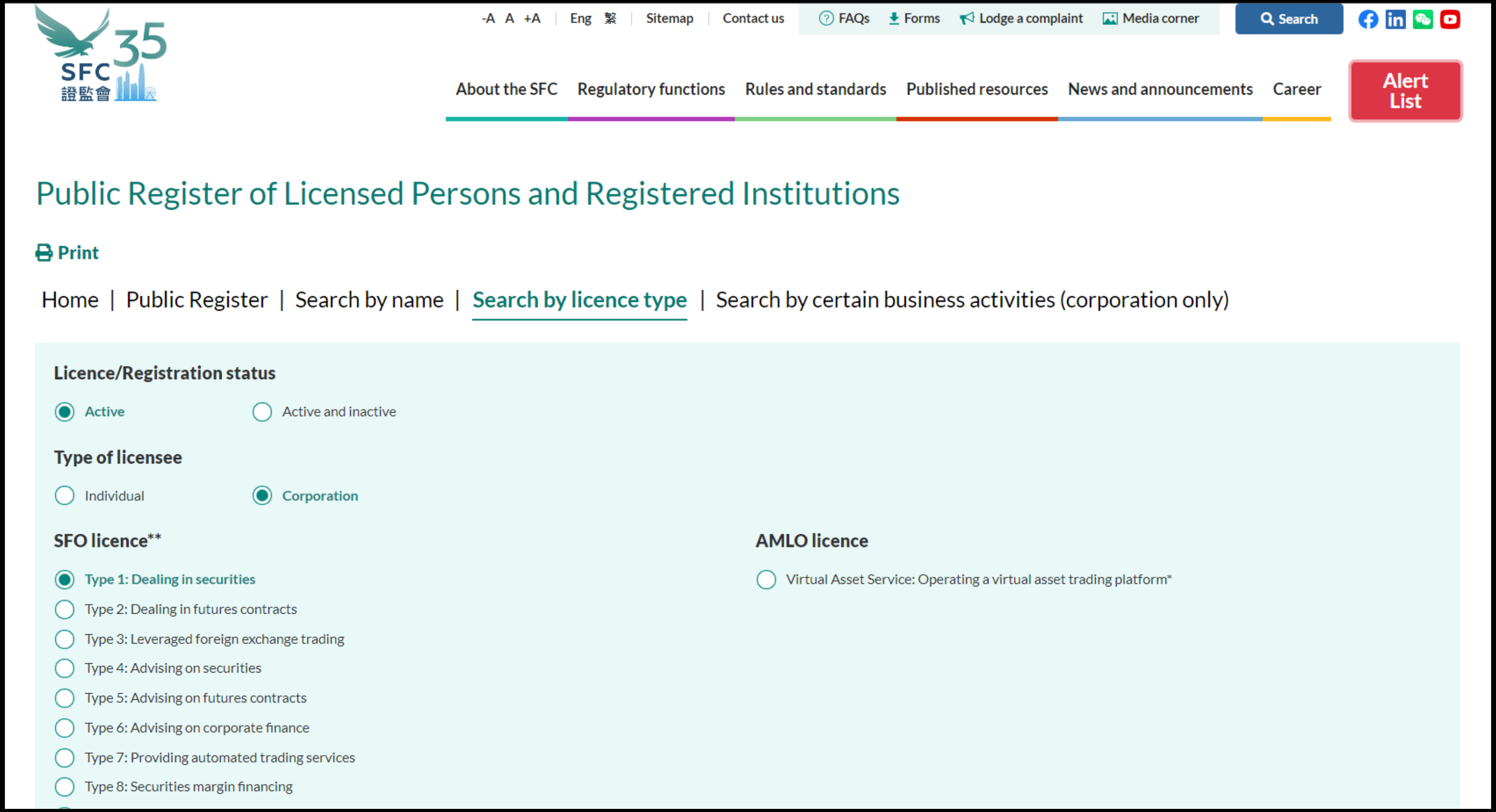
HKSFC License Requirement

- Type 1: Dealing in securities
- Type 2: Dealing in futures contracts
- Type 3: Leveraged foreign exchange trading
- Type 4: Advising on securities
- Type 5: Advising on futures contracts
- Type 6: Advising on corporate finance
- Type 7: Providing automated trading services
- Type 8: Securities margin financing
- Type 9: Asset management
- Type 10: Providing credit rating services
- Type 11: Dealing in OTC derivative products or advising on OTC derivative products
- Type 12: Providing client clearing services for OTC derivative transactions
- Type 13: Providing depositary services for relevant CISs

Reference: https://www.sfc.hk/-/media/EN/assets/components/codes/files-current/web/licensing-handbook/LIC-Handbook-2025-Eng.pdf?rev=77b73d8a0caf4663a2c8e8c2aeab6d47

HKSFC Licensed Brokers

- Reference: https://apps.sfc.hk/publicregWeb/searchByRa?locale=en



5. Customer Support

- Availability of Support
  - 24/7 support can help resolve issues quickly
- Educational Resources
  - Quality brokers offer training materials, webinars, and articles to enhance trader skills.
  - Comprehensive documentation about its user interface, system functionalities, etc

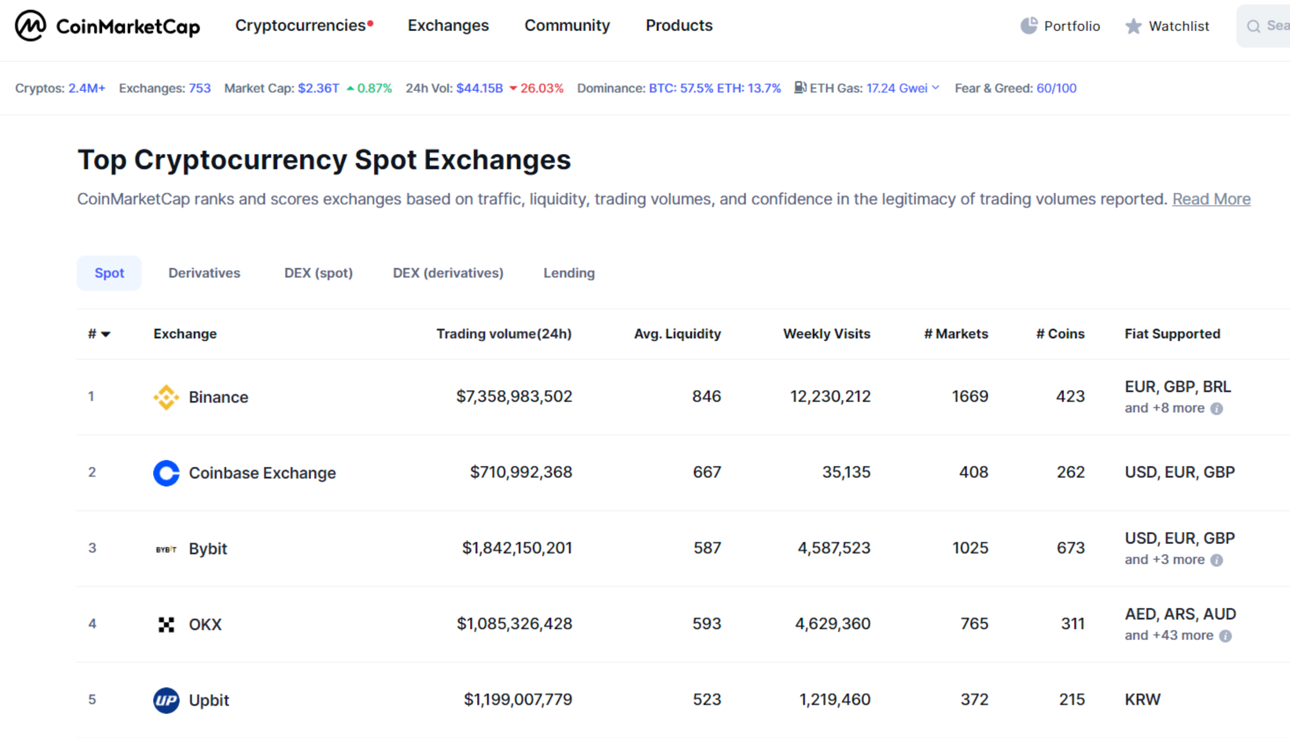
6. Reliability and reputation

- Users review and comments on forum, social media, etc
- The history of the brokerage firm (eg. company's age, industrial awards, number of customers, etc)
- Listed companies are more reliable due to responsibility to regularly disclose its business and financials

Useful Resources - CoinMarketCap

CoinMarketCap is a data platform to keep track on crypto market

- Centralized Exchanges
  - Spot: https://coinmarketcap.com/rankings/exchanges/
  - Derivatives: https://coinmarketcap.com/rankings/exchanges/derivatives/
- Decentralized Exchanges
  - Spot: https://coinmarketcap.com/rankings/exchanges/dex/?type=spot
  - Derivatives: https://coinmarketcap.com/rankings/exchanges/dex/?type=derivatives


CoinMarketCap Cryptocurrencies Exchanges Community Products Portfolio Watchlist

Cryptos: 2.4M+ Exchanges: 753 Market Cap: $2.36T 0.87% 24h Vol: $44.15B 26.03% Dominance: BTC: 57.5% ETH: 13.7% ETH Gas: 17.24 Gwei Fear & Greed: 60/100

Top Cryptocurrency Spot Exchanges

CoinMarketCap ranks and scores exchanges based on traffic, liquidity, trading volumes, and confidence in the legitimacy of trading volumes reported. Read More

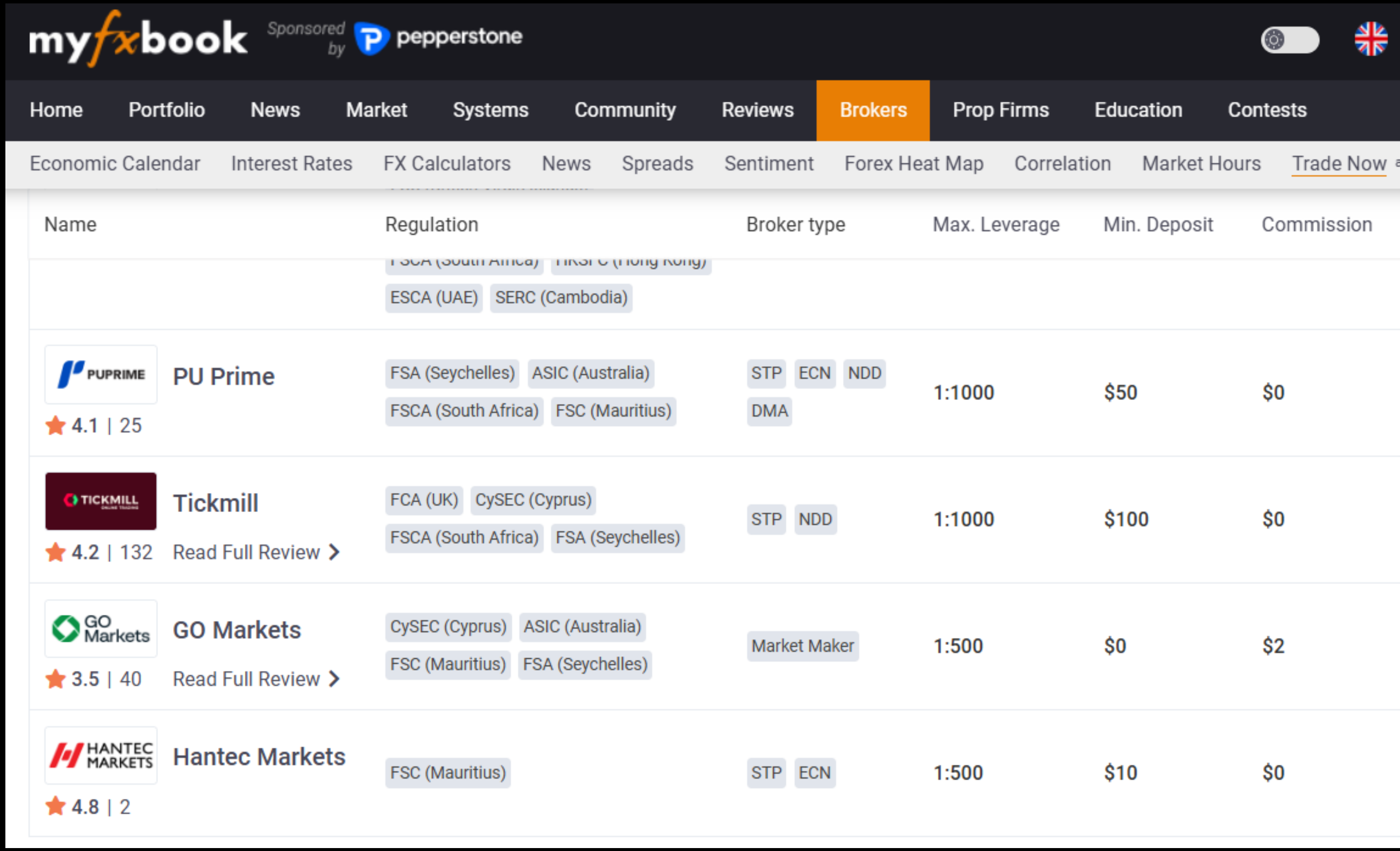
Spot Derivatives DEX (spot) DEX (derivatives) Lending

#	Exchange	Trading volume(24h)	Avg. Liquidity	Weekly Visits	# Markets	# Coins	Fiat Supported
1	Binance	$7,358,983,502	846	12,230,212	1669	423	EUR, GBP, BRL and +8 more
2	Coinbase Exchange	$710,992,368	667	35,135	408	262	USD, EUR, GBP
3	Bybit	$1,842,150,201	587	4,587,523	1025	673	USD, EUR, GBP and +3 more
4	OKX	$1,085,326,428	593	4,629,360	765	311	AED, ARS, AUD and +43 more
5	Upbit	$1,199,007,779	523	1,219,460	372	215	KRW

Useful Resources - MyFxBook

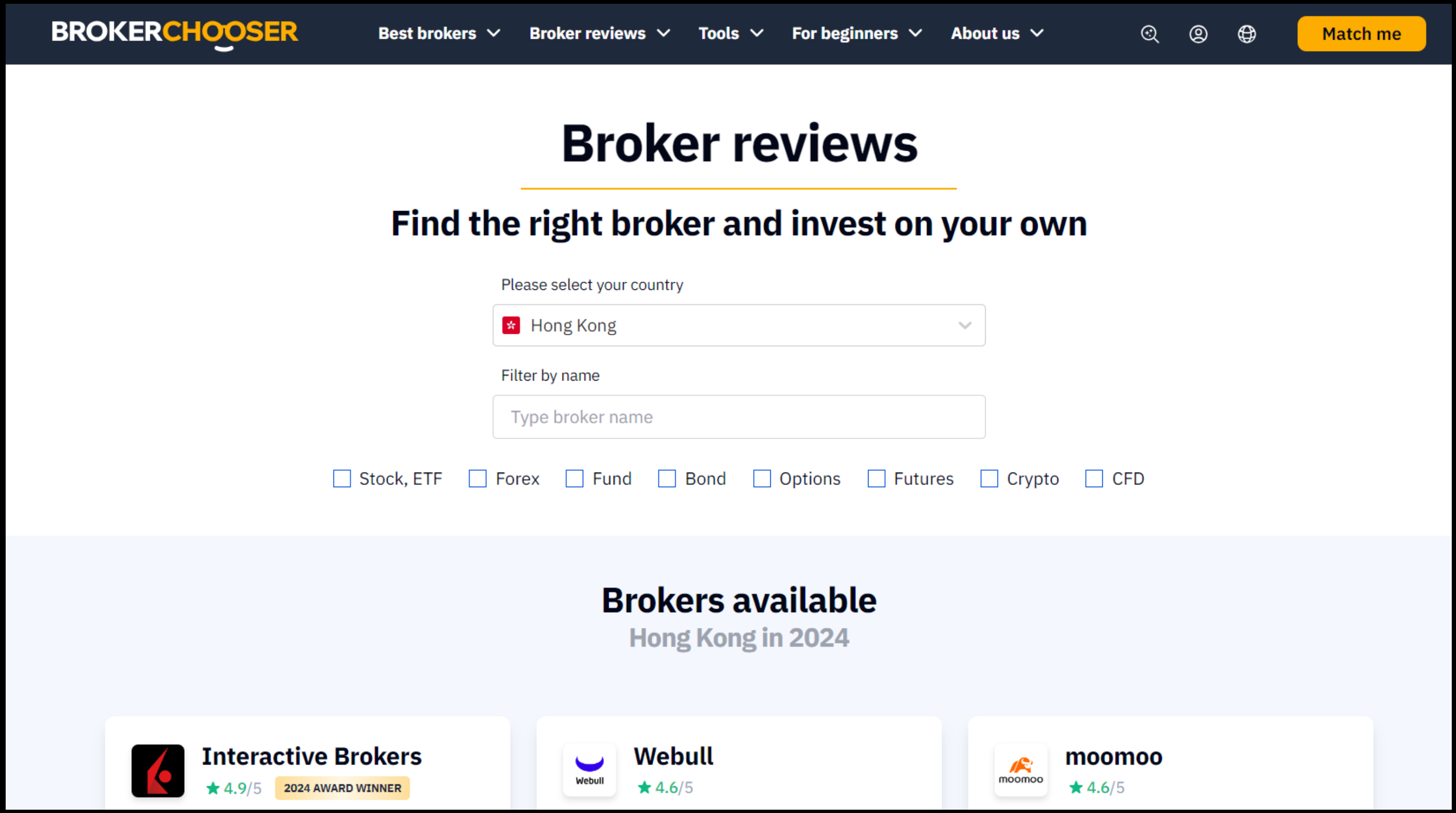
MyFxBook is a data platform to keep track on forex market

- License: https://www.myfxbook.com/forex-brokers
- Quote: https://www.myfxbook.com/forex-broker-quotes
- Spread: https://www.myfxbook.com/forex-broker-spreads
- Volume: https://www.myfxbook.com/forex-broker-volume
- Swaps Fee: https://www.myfxbook.com/forex-broker-swapsv



Useful Resources - BrokerChooser

- https://brokerchooser.com/broker-reviews



Best Practices in Broker Selection

- **Research and Reviews:**
  - Use trader forums and reviews
- **Demo Accounts:**
  - Test the platform before committing
- **Stay Updated:**
  - Regularly review broker performance and fees

FTX Case Study

FTX Case Study

- Background
  - FTX was the global 3rd largest cryptocurrency exchange in 2022.
  - High liquidity and a wide range of trading instruments.
  - Rapid growth led to large investments and user deposits.

FTX Case Study

- Background
    - Sam Bankman-Fried, the CEO of FTX, moved clients’ deposit to his affiliated trading firm, Alameda Research for speculative cryptocurrency trading
    - This situation was reported by journalists in early 2022 Nov, and therefore a surge of customer withdrawals due to concerns over the mismanagement and lack of transparency in financial practices.
    - It pushed FTX and Alameda into bankruptcy and shook the volatile crypto market.
    - In December 2022, the U.S. government brought civil and criminal charges against Sam Bankman-Fried and top executives for misappropriating over $8 billion in customer deposits.
    - In March 2024, Bankman-Fried was convicted and sentenced to 25 years in prison for stealing $8 billion from customers.

FTX Case Study - Counterparty risk

- What is counterparty risk?
  - The risk that the other party in a transaction may default on their obligations.
  - You have a profitable strategy and make lot of money, but unable to withdraw due to the broker’s financing problem

- Counterparty Risk Exposed
  - FTX filed for bankruptcy in November 2022.
  - Customers lost billions due to misallocated funds and fraudulent practices
  - Many traders were unable to access their funds.
  - Highlighted the dangers of relying on a broker without sufficient oversight.

FTX Case Study

Lessons Learned

1. Due Diligence
   - Always research a broker's reputation and regulatory compliance
   - Look for transparency in operations and financial health
2. Regulation Matters
   - Choose brokers regulated by reputable authorities to reduce risk
   - Understand the protections offered under those regulations
3. Diversification
   - Avoid putting all money with a single broker
   - Spread risk across multiple platforms where possible

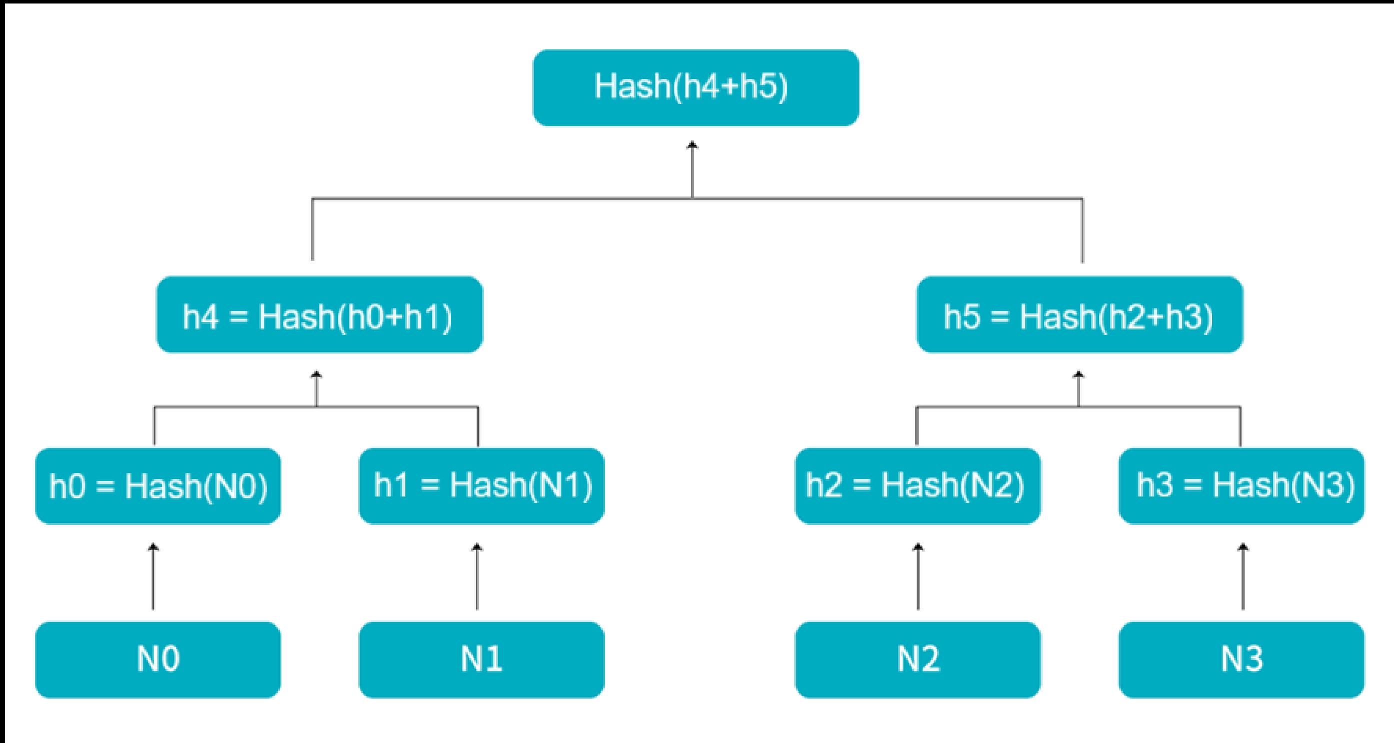
Proof-Of-Reserves (PoR)

Proof-Of-Reserves (PoR)

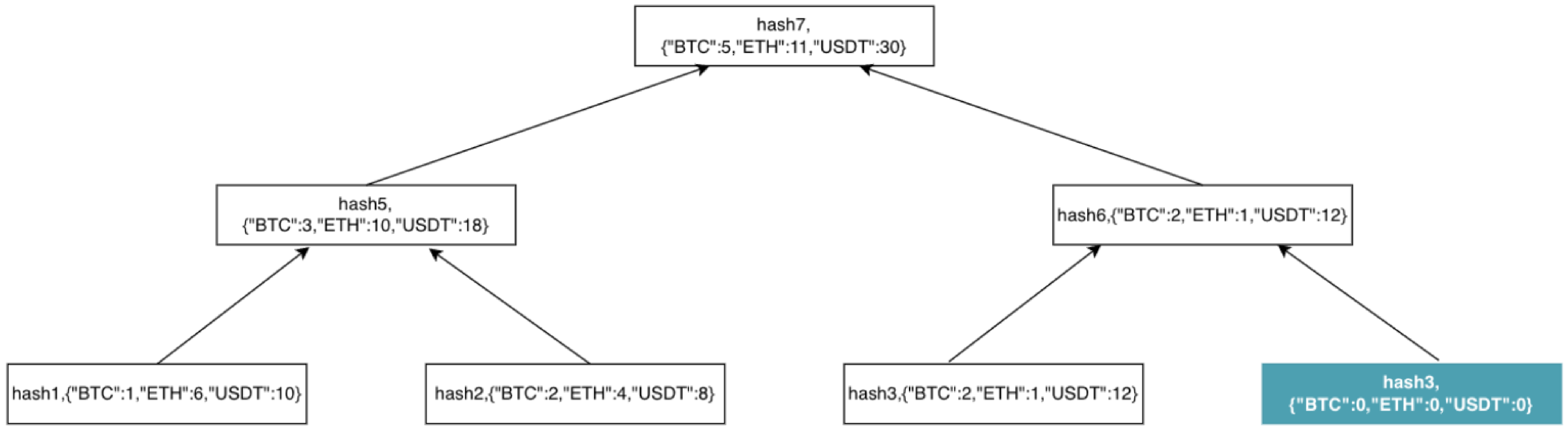
- PoR is a method used by financial institutions and exchanges to prove they hold sufficient reserves to back their liabilities.
- The purpose is to enhance trust and transparency with investors.
- For traditional brokerage firms, it requires third party auditing firm to verify and accredit the figures.
- For crypto exchanges, Merkle Tree with blockchain is commonly used for PoR purpose nowadays.

Merkle Tree

- A Merkle Tree is a data structure that allows efficient and secure verification of the integrity of large data sets.
- Structure:
  - Leaf nodes represent data (eg. transactions, positions, etc).
  - Non-leaf nodes are hashes of their child nodes (eg. SHA-256)
- Key Feature:
  - Allows verification of data integrity without revealing the entire dataset.



Merkle Tree



hash7,
{"BTC":5,"ETH":11,"USDT":30}
hash5,
{"BTC":3,"ETH":10,"USDT":18}
hash6,{"BTC":2,"ETH":1,"USDT":12}
hash1,{"BTC":1,"ETH":6,"USDT":10}
hash2,{"BTC":2,"ETH":4,"USDT":8}
hash3,{"BTC":2,"ETH":1,"USDT":12}
hash3,
{"BTC":0,"ETH":0,"USDT":0}

How Merkle Trees Work

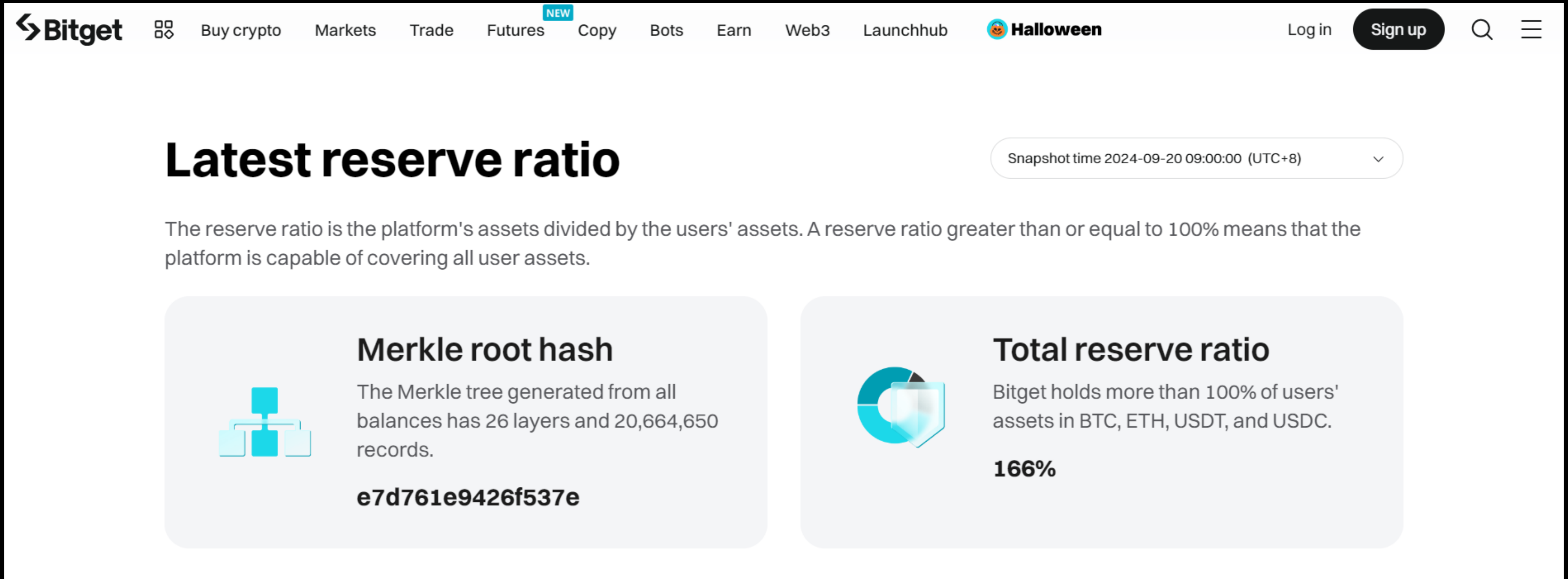
1. Data is hashed into leaf nodes.
2. Leaf nodes are paired and hashed to form parent nodes.
3. This process continues until a single hash (Merkle Root) is generated.
4. The Merkle Root serves as a compact representation of all data.

How Proof-of-Reserves Works

- **Audit Process:**
  - The institution publishes a Merkle Tree of its customer balances.
  - A third-party auditor verifies the accuracy of the Merkle Root.
- **Verification:**
  - Customers can verify that their balance is included in the Merkle Tree without revealing sensitive information.
  - Ensures total reserves match or exceed customer liabilities.

Reserve Ratio

- It measures the company’s asset over liability (i.e. total client’s deposit)
- The ratio greater than 1 means the client’s deposit is kept within the exchange
- Reference: https://www.bitget.com/proof-of-reserves



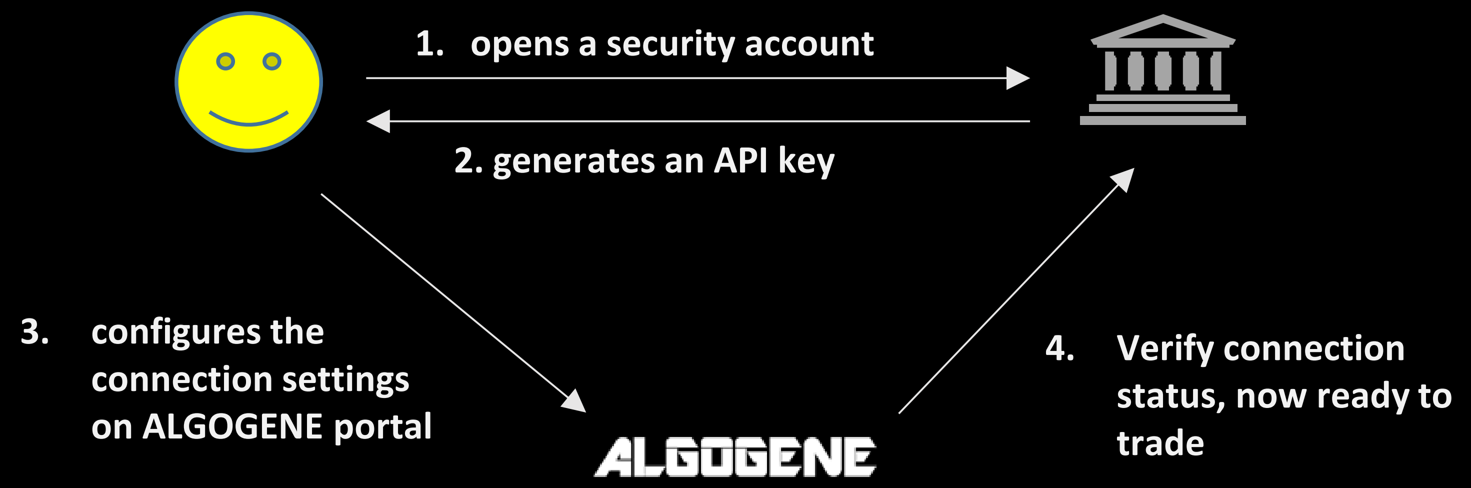
Benefits of Using Merkle Trees in PoR

- **Data Integrity**: Ensures that the published data has not been tampered with.
- **Privacy**: Customers can verify their holdings without exposing personal data.
- **Efficiency**: Allows quick verification of large datasets.
- **Timeliness**: Unlike traditional brokers which publish once per quarter, many crypto exchanges publish the data online every hour

Algo Deployment with ALGOGENE

Functionalities

- Trade and manage multiple broker accounts within a single platform
- Seamlessly deploy an algo script to multiple broker accounts without code conversion
- Real-time data access from multiple brokers/exchanges



Available brokers/exchanges

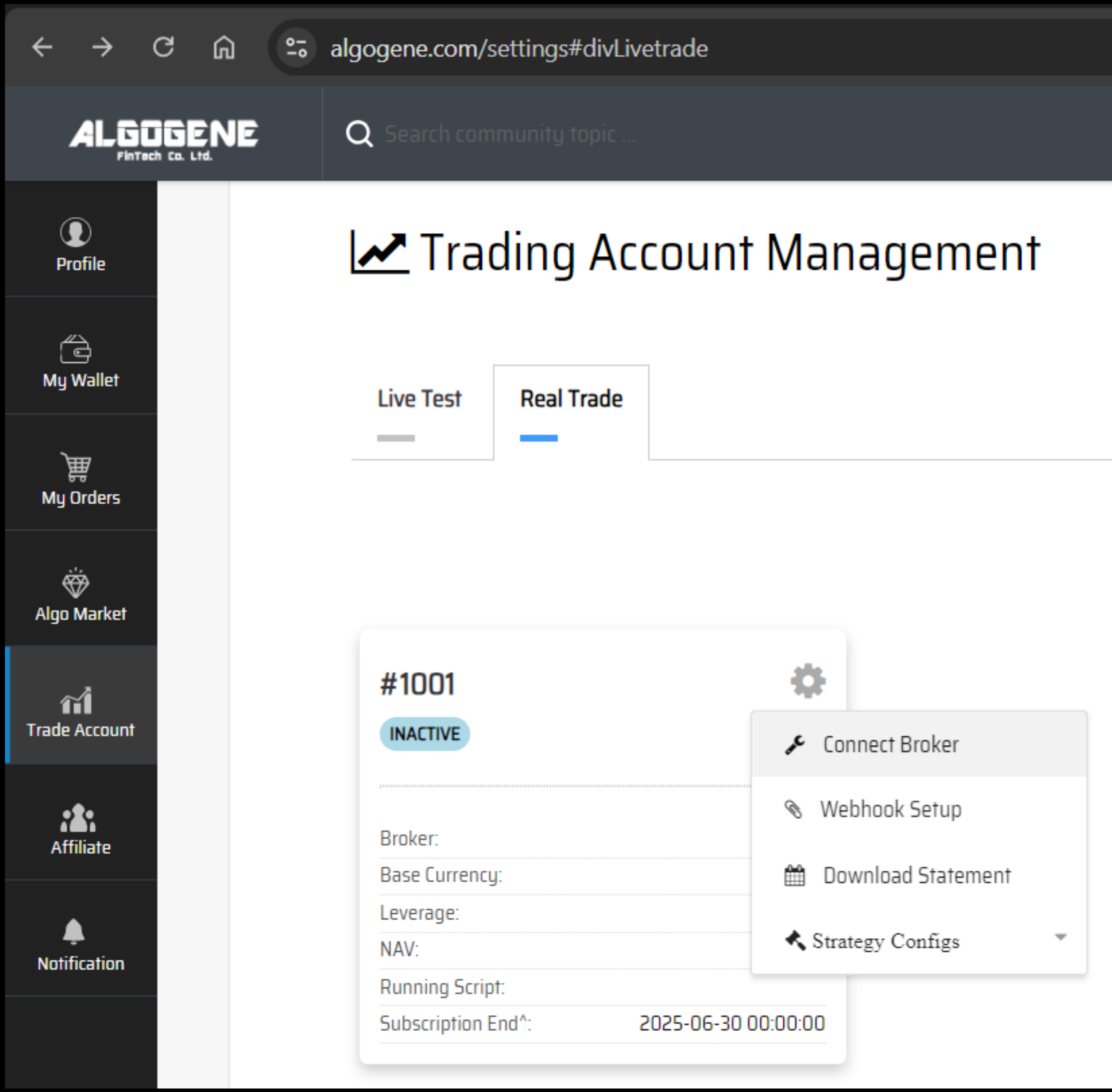
- Supports 30+ brokerage firms and crypto exchanges including but not limited to below
- Lifetime 5-10% commission discount if register using the partnership links

Alpaca	https://algogene.com/community/post/92
Binance	https://algogene.com/community/post/55
Bitget	https://algogene.com/community/post/99
Bybit	https://algogene.com/community/post/113
Exness	https://algogene.com/community/post/53
GO Markets	https://algogene.com/community/post/260
IG	https://algogene.com/community/post/15
Interactive Brokers	https://algogene.com/community/post/25
KuCoin	https://algogene.com/community/post/146
OKX	https://algogene.com/community/post/104

Reference: https://algogene.com/community/post/16

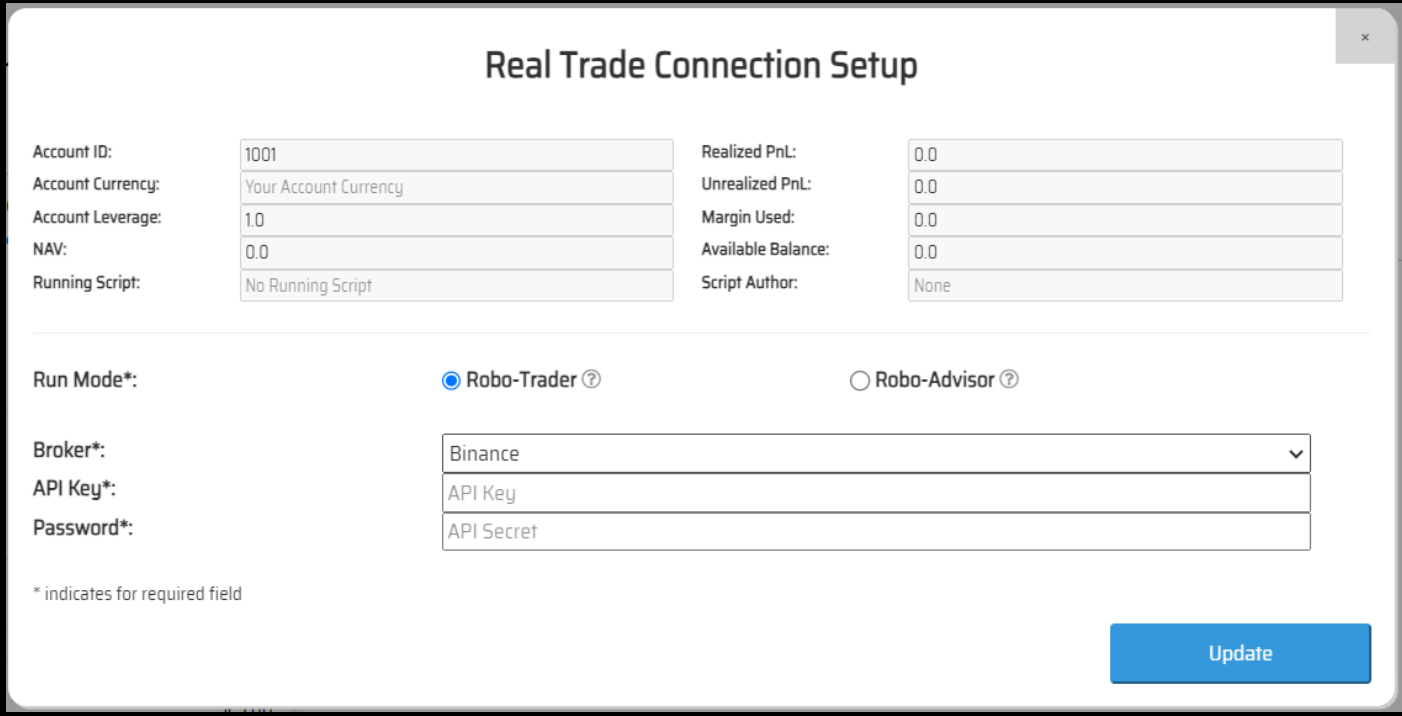
Configuration steps

1. After login the platform, go to [Settings] page
2. Go to [Trade Account] section
3. Choose “**Live Test**” or “**Real Trade**”
4. Click “Connect Broker”



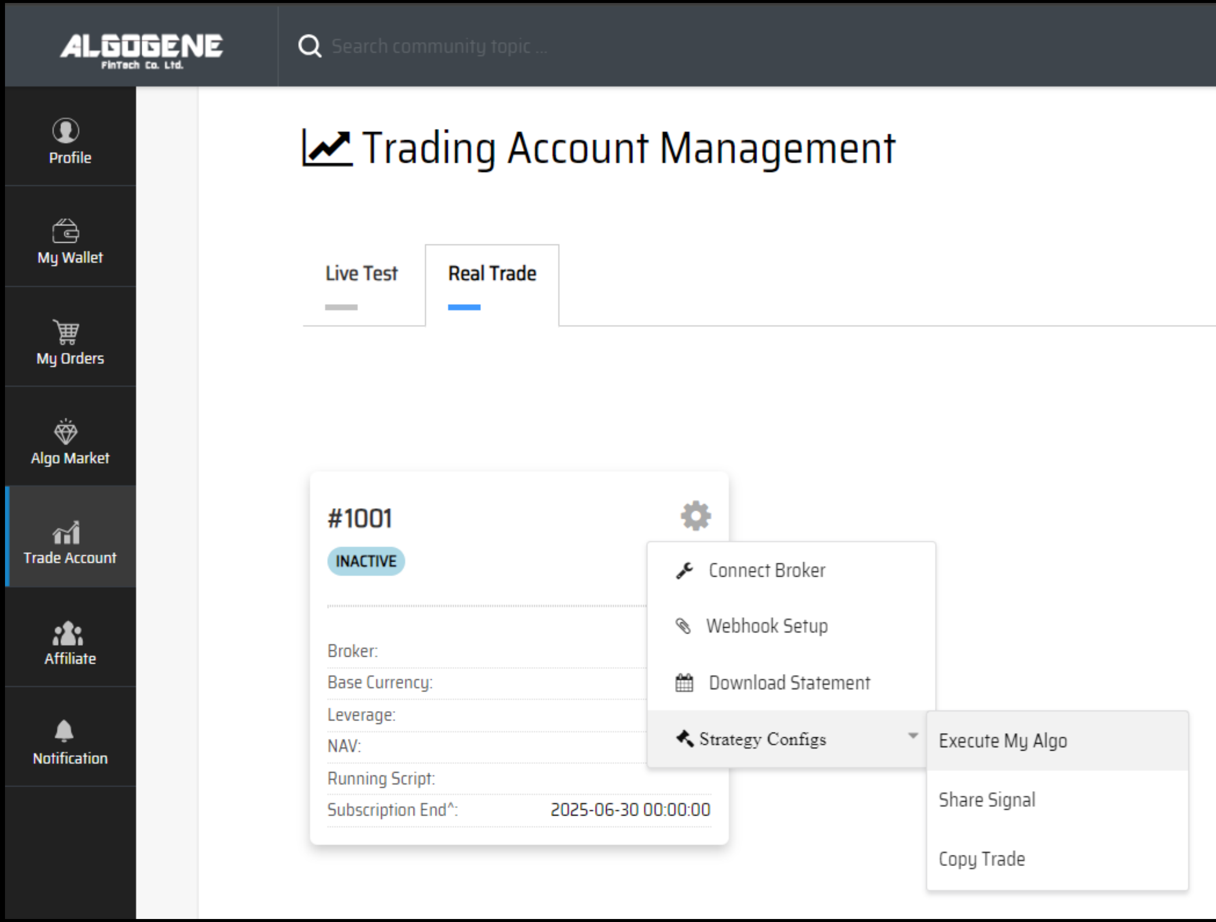
Configuration steps

5. Select your broker
6. Update with your API info
7. After successful connection, the broker's account info will be synchronized on the platform



Configuration steps

8. Click “**Strategy Configs**”, then “**Execute My Algo**” to deploy a backtest script to the connected broker account



Market Tricks

Understand the business world

- Some traders and investment firms may intentionally present their performance in certain ways to attract investment or increase sales
- Examples:
  - Misleading metrics
  - Selective presentation of data
  - Psychological manipulation

Misleading Metrics - Example

- Winning Rate Definition:
  1. the percentage of profitable trades, measured from a round order perspective (misleading)
  2. the percentage of trading day that end up with a gain (more appropriate)
- Using the misleading definition
  - Tricks:
    - Firms may highlight a high winning rate without disclosing average losses on losing trades.
    - Only close profitable trades but leave losing trades open to achieve a high winning rate
  - Impact: a high winning rate does not necessarily indicate overall profitability.
- Definition 2 is more applicable to both trade-based and position-based system

Winning Rate - Example

- This trade has 4 consecutive loss days, but finally ends up with a big gain.
- Using definition 1, as there is only 1 profitable trade, so winning rate is 100%
- Using definition 2, the winning rate is 1/5 = 20%

Day	Closing Price	Daily PnL
0	110	0
1	108	-2
2	105	-3
3	104	-1
4	100	-4
5	130	30
Total PnL		**20**

Selective Presentation – Example 1

- The formula of Sharpe ratio is

assume risk free rate = 0

$$SharpeRatio = \frac{Annualized\ Return}{Annualized\ Volatility}$$

- **Trick**: Firms may present a high Sharpe Ratio by using short time frames with low volatility.
- **Impact**: Misleading risk assessment; may not reflect long-term performance.

Selective Presentation – Example 2

- The formula of Sharpe ratio is $\boldsymbol{SharpeRatio = \frac{Annualized\ Return}{Annualized\ Volatility}}$

assume risk free rate = 0

- It can be calculated using different ways.
  - For daily data, $SharpeRatio = \frac{Daily\ Return}{Daily\ Volatility}\sqrt{252}$
  - For weekly data, $SharpeRatio = \frac{Weekly\ Return}{Weekly\ Volatility}\sqrt{52}$
  - For monthly data, $SharpeRatio = \frac{Monthly\ Return}{Monthly\ Volatility}\sqrt{12}$
- **Trick**: Firms presents the one that produce the highest value.
- **Impact**: Not an apple-to-apple comparison between different investment products

Psychological Manipulation: "Fear of Missing Out" (FOMO)

- Leverage the emotional response of potential investors to create urgency and drive decisions.
- Example:
  - Promotional Campaign:
    - Claim: "Join the smart investors who gained 30% in just six months!“
  - Tactics Used:
    - Urgency: Limited-time offers or exclusive access to "top-performing" funds.
    - Social Proof: Showcasing testimonials from satisfied clients who experienced high returns.
    - Imagery: Using visuals of luxury lifestyles (e.g., cars, vacations) to evoke desire.
- Impact:
  - Investors may rush to invest out of fear of missing potential gains.
  - Emotional decisions often overshadow logical analysis, leading to investments in high-risk or unsuitable products.

Conclusion & Advices

- Always look beyond the surface metrics.
- Consider full context: time frames, benchmarks, and risk measures.
- Perform due diligence and seek clarity on performance metrics.
- Remain rational and avoid rush investment decisions driven by emotion.

Key Takeaways

- 3 strategy optimization methods, including brute force, gradient decent, genetic algorithm
- Choose the parameter set that yield a robust output is more appropriate than choosing the optimal solution
- There are various type of investment funds in the market designed to cater for different investor needs.
- Hedge funds deploy a large number of uncorrelated strategies to achieve long winning track records
- Important things to consider when choosing a trading broker
- Common sales tricks used by traders/ investment firms